Programming and Data Structures - II (CS3201)

Lecture 3

Kripabandhu Ghosh

CDS, IISER Kolkata

HEAP

Heap

Definition

Heap is a Binary Tree that satisfies the following properties:

- **Shape Property**:
  - All levels, except the deepest, are fully filled
  - The deepest level is filled from left to right

Heap

Definition

Heap is a Binary Tree that satisfies the following properties:

- **Shape Property**:
  - All levels, except the deepest, are fully filled
  - The deepest level is filled from left to right

    Complete Binary tree

Heap

Definition

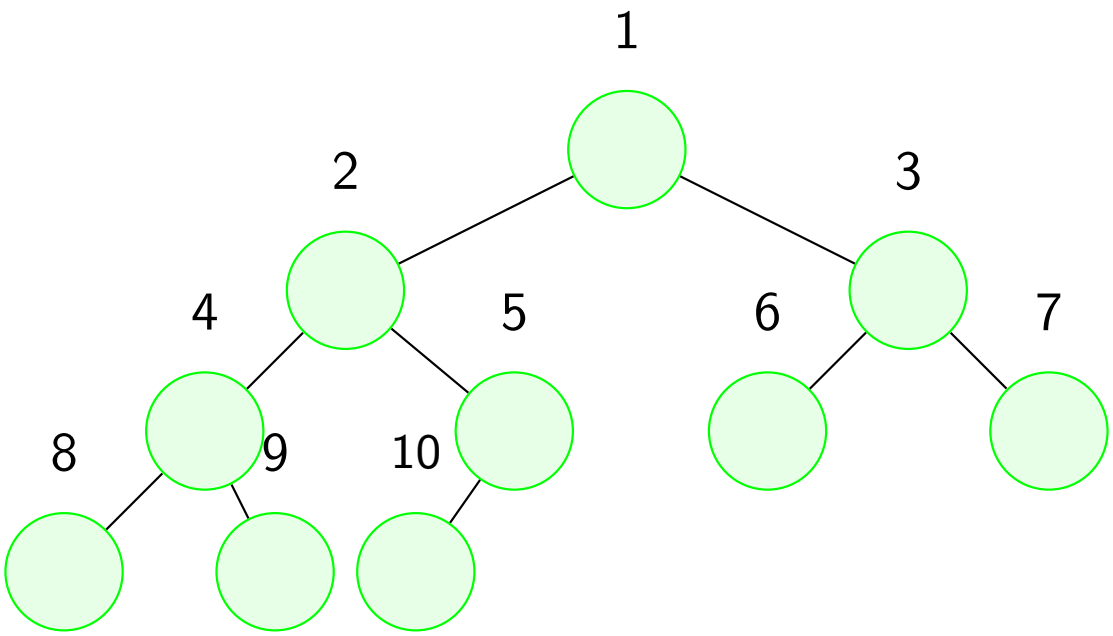
Heap is a Binary Tree that satisfies the following properties:

- **Shape Property**:
  - All levels, except the deepest, are fully filled
  - The deepest level is filled from left to right
    Complete Binary tree
- **Heap Property**:
  - (Max heap) Value of a node $\geq$ Value of its children
  - (Min heap) Value of a node $\leq$ Value of its children

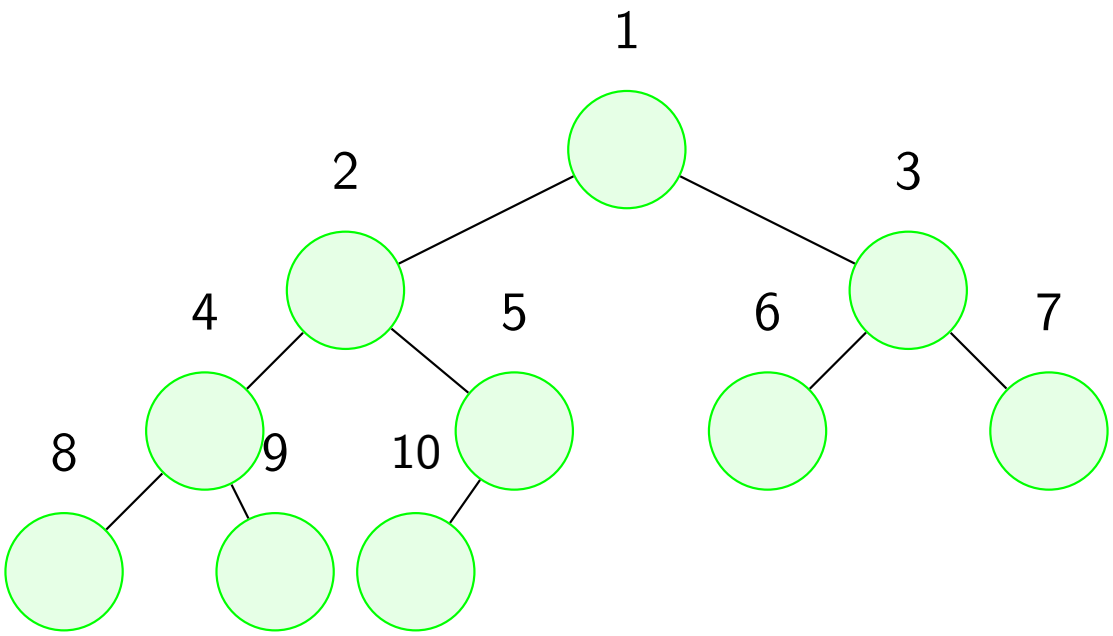
Heap (also Heapsort) was introduced by John William Joseph Williams in 1964.[a]

[a] https://en.wikipedia.org/wiki/Heap_(data_structure)

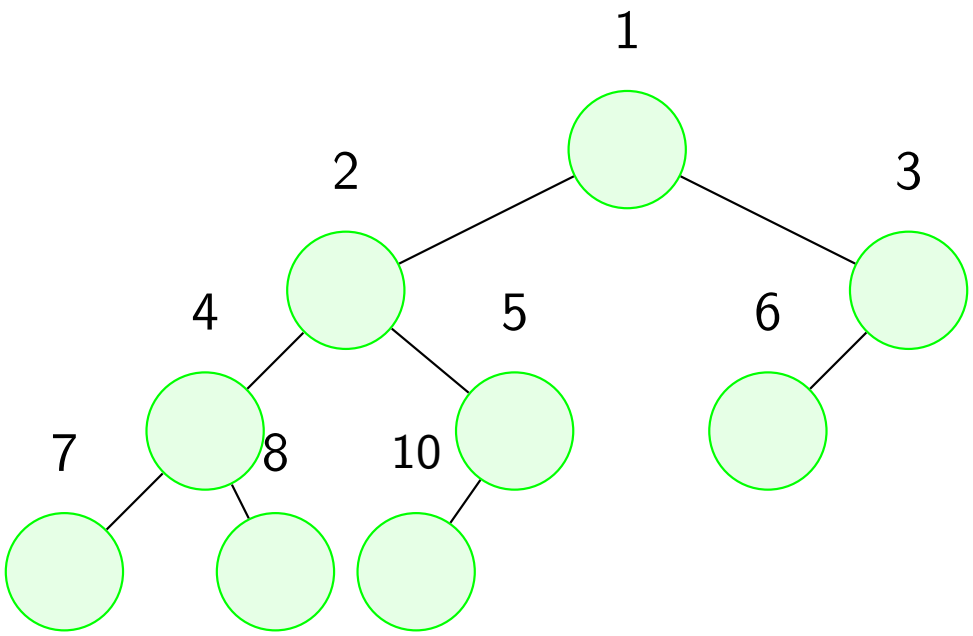
Shape property: ?



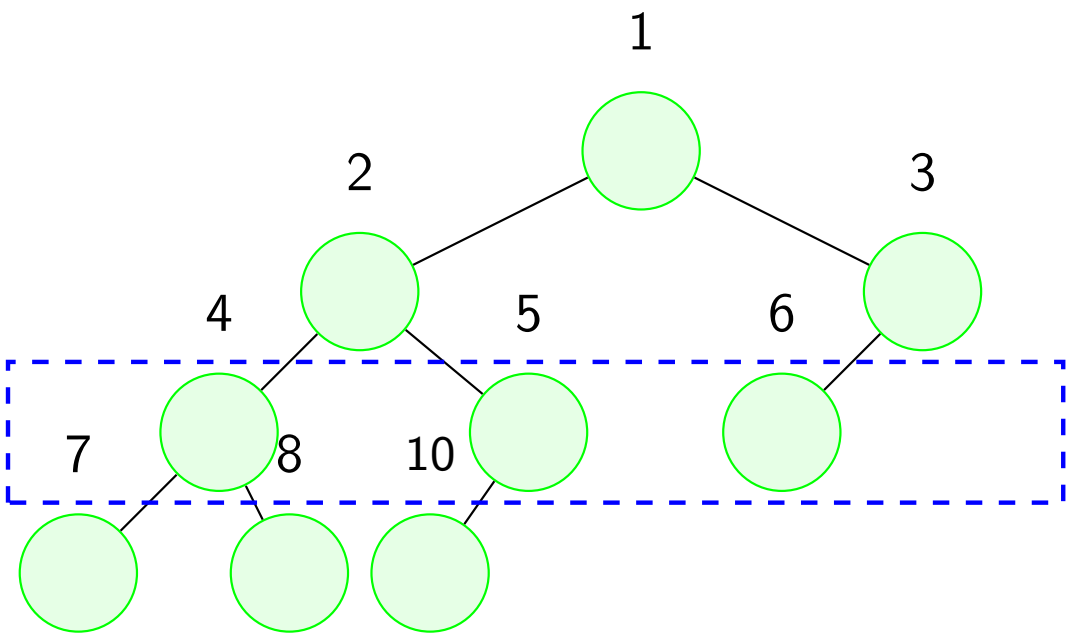
Shape property: Satisfied!!



Shape property: ?

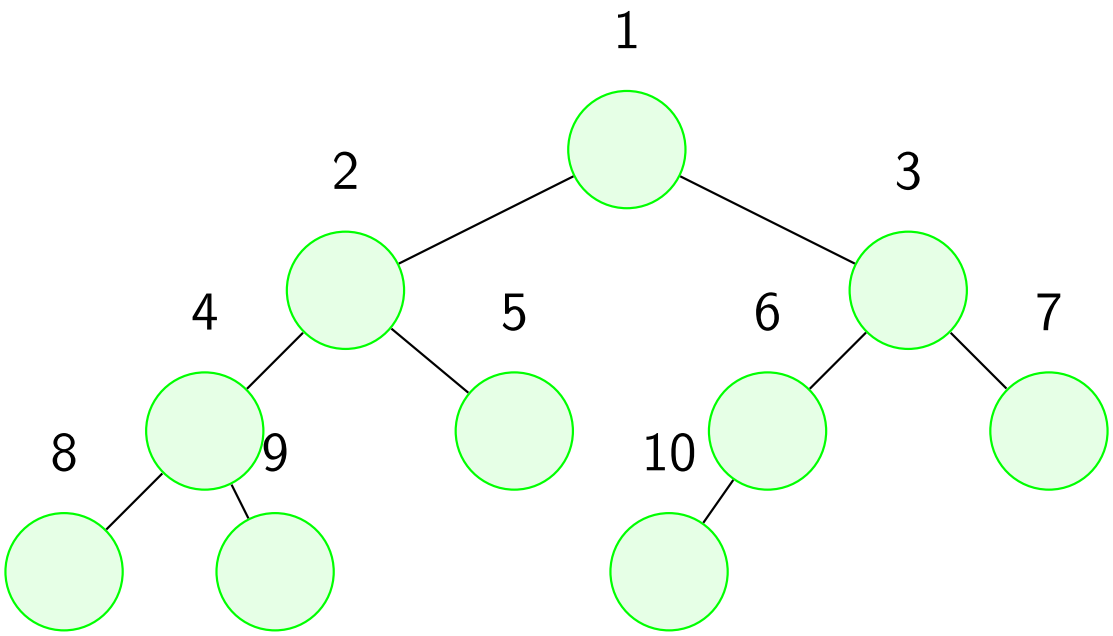


Shape property: Violated!!

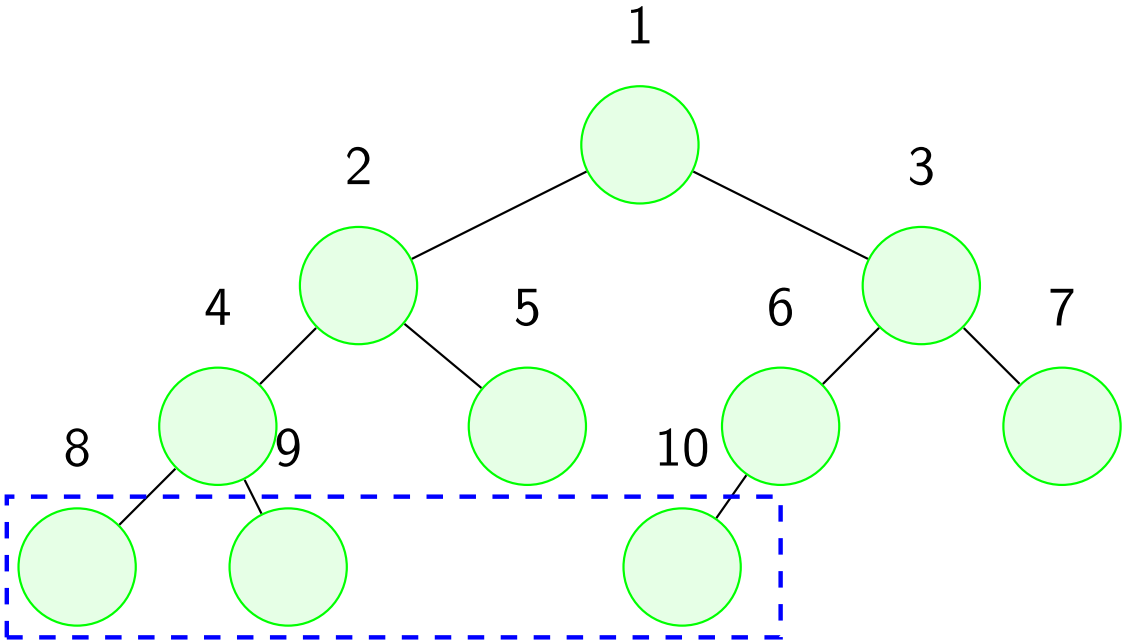


Level (not the deepest) NOT fully filled !!!

Shape property: ?



Shape property: Violated!!



Deepest level NOT filled from left to right !!!

Array implementation of Heap

Heap array A

- A.*length*: Number of elements in array A
- A.*heap-size*: Number of elements of the heap stored in A
- $0 \leq$ A.*length* $\leq$ A.*heap-size*

Root at A[1]

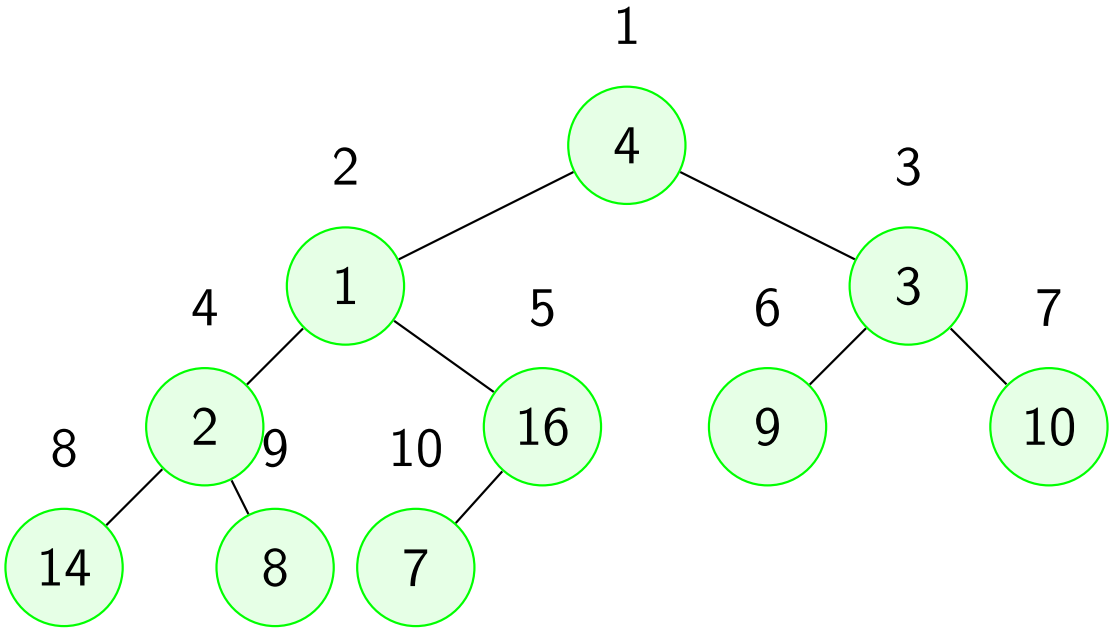
Node at index i

- Left child (LEFT(i)) has index $2i$
- Right child (RIGHT(i)) has index $2i + 1$
- Parent (PARENT(i)) at $\lfloor i/2 \rfloor$

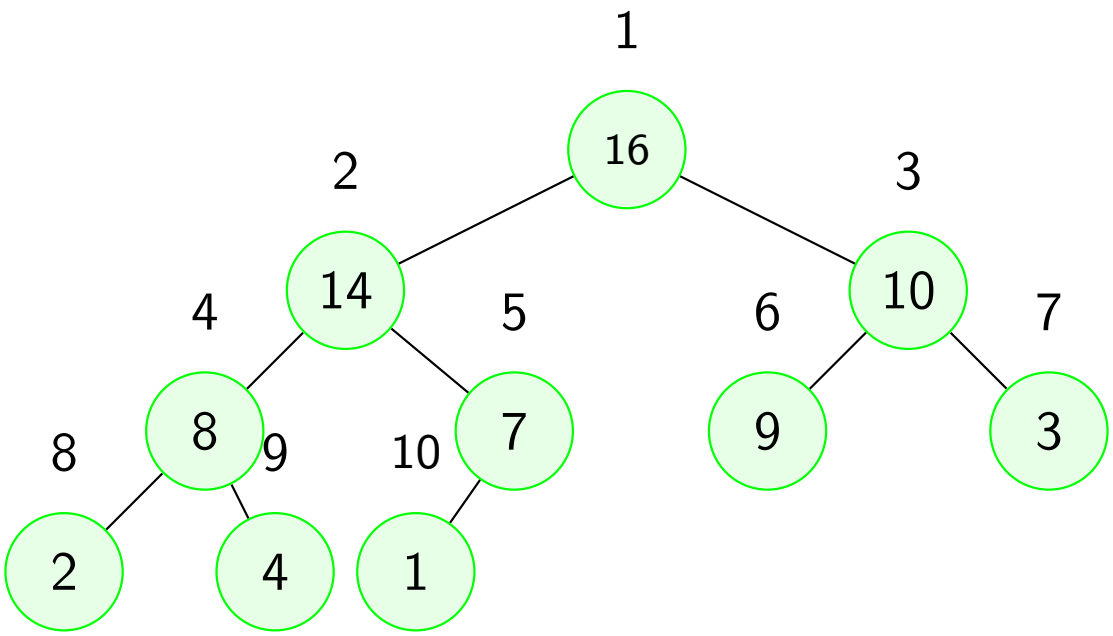
For an n-element heap, the leaves are at the nodes indexed by $\lfloor n/2 \rfloor + 1$, $\lfloor n/2 \rfloor + 2$, ..., n

Array implementation of Heap (contd.)

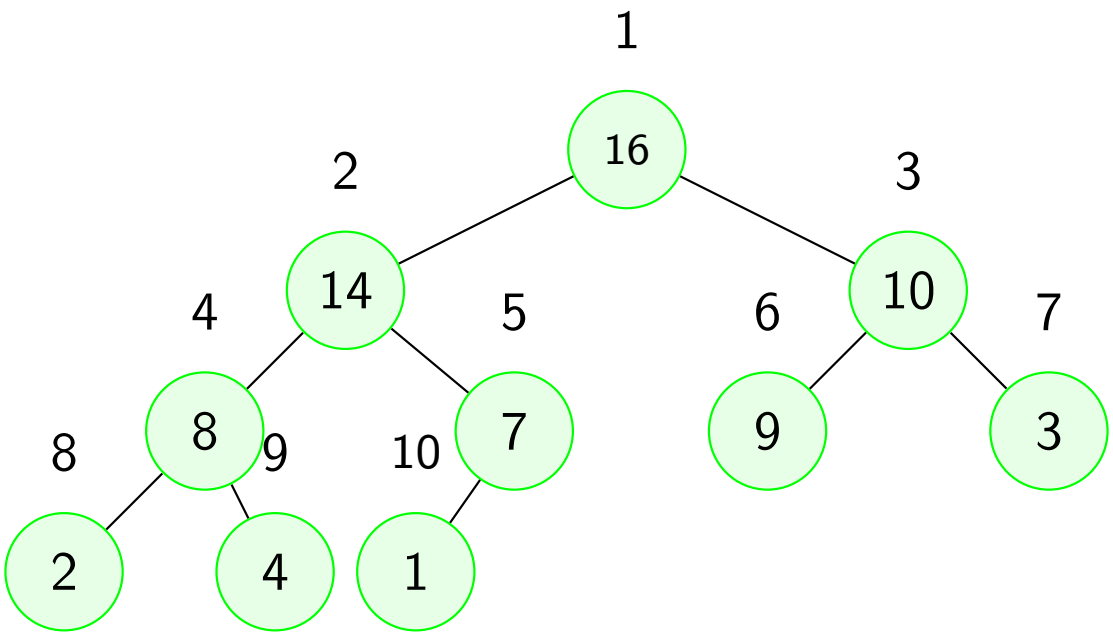
	1	2	3	4	5	6	7	8	9	10
A	4	1	3	2	16	9	10	14	8	7



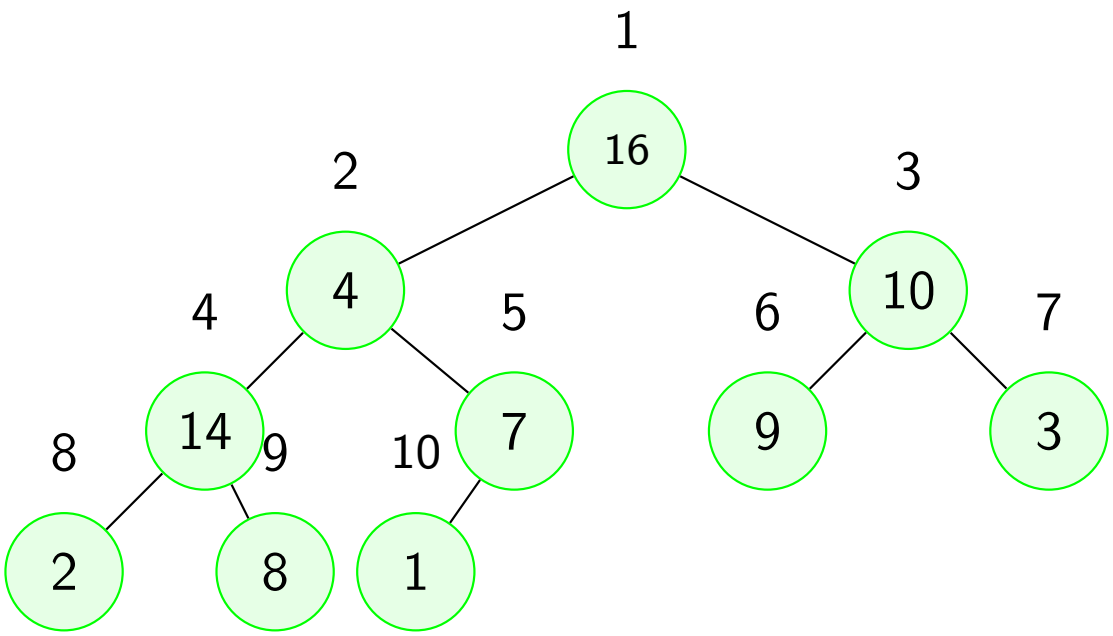
(Max) Heap Property: ?



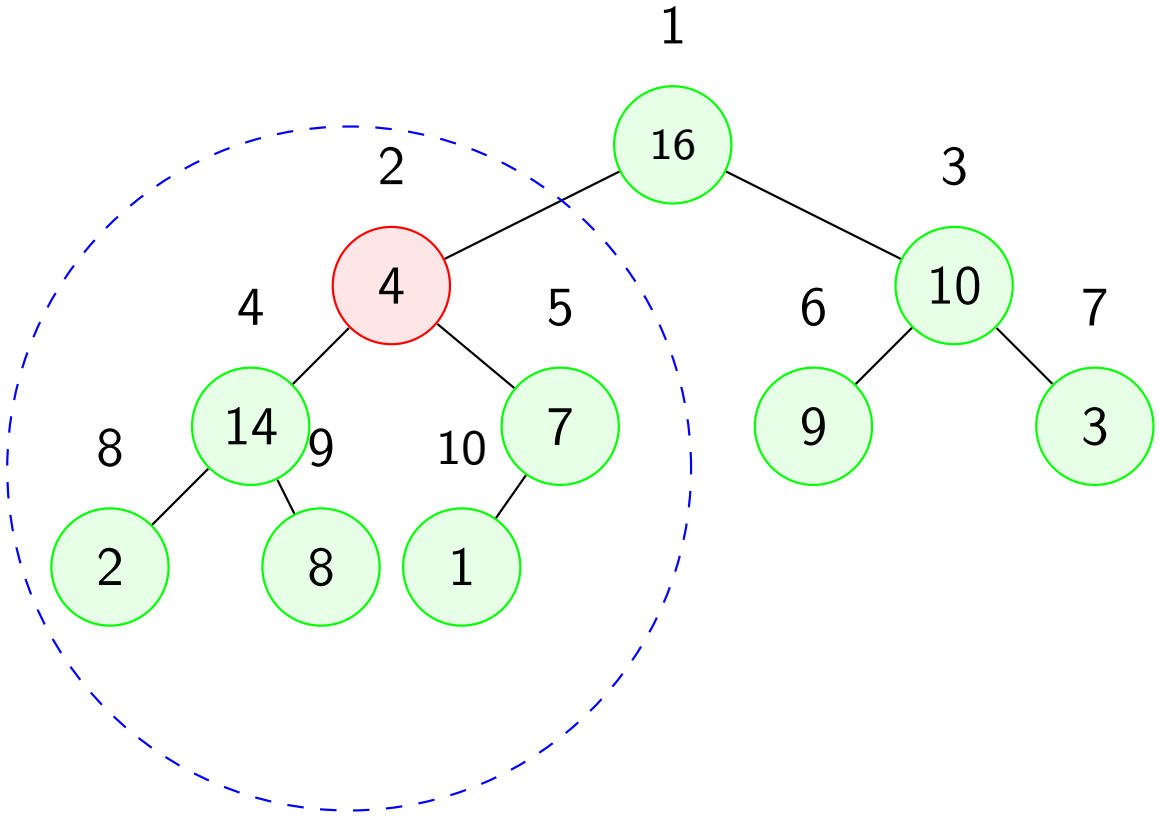
(Max) Heap Property: Satisfied!



(Max) Heap Property: ?



(Max) Heap Property: Violated!!



Node no. 2 (value 4) has value less than those of its children !!

Maintaining (Max) Heap Property

- **Max-heapify** applied
- Input: A and index i
- **Assumption**: Left and Right subtrees of A[i] are max-heaps
- A[i] may be smaller than its children

Maintaining (Max) Heap Property

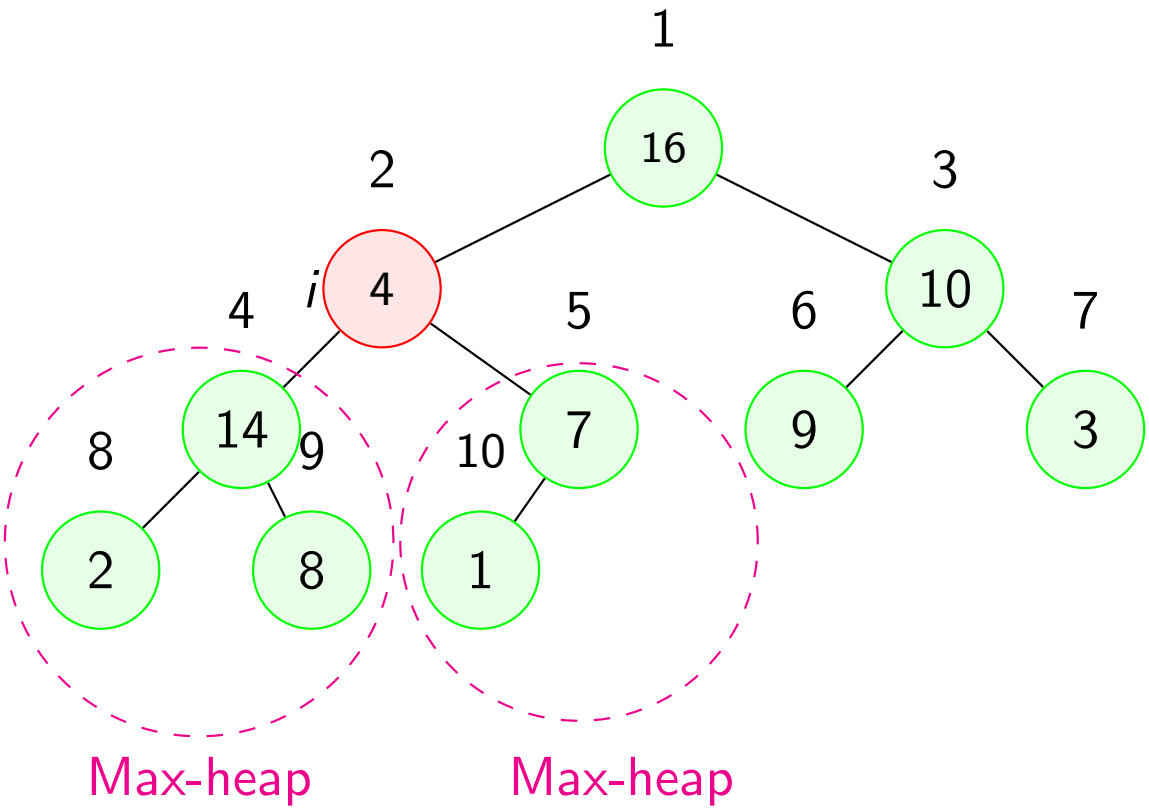
- **Max-heapify** applied
- Input: A and index i
- **Assumption**: Left and Right subtrees of A[i] are max-heaps
- A[i] may be smaller than its children $\Rightarrow$ Violating Max-heap property
- Appropriate action taken to ensure the tree rooted at A[i] becomes a Max-heap

Max-heapify

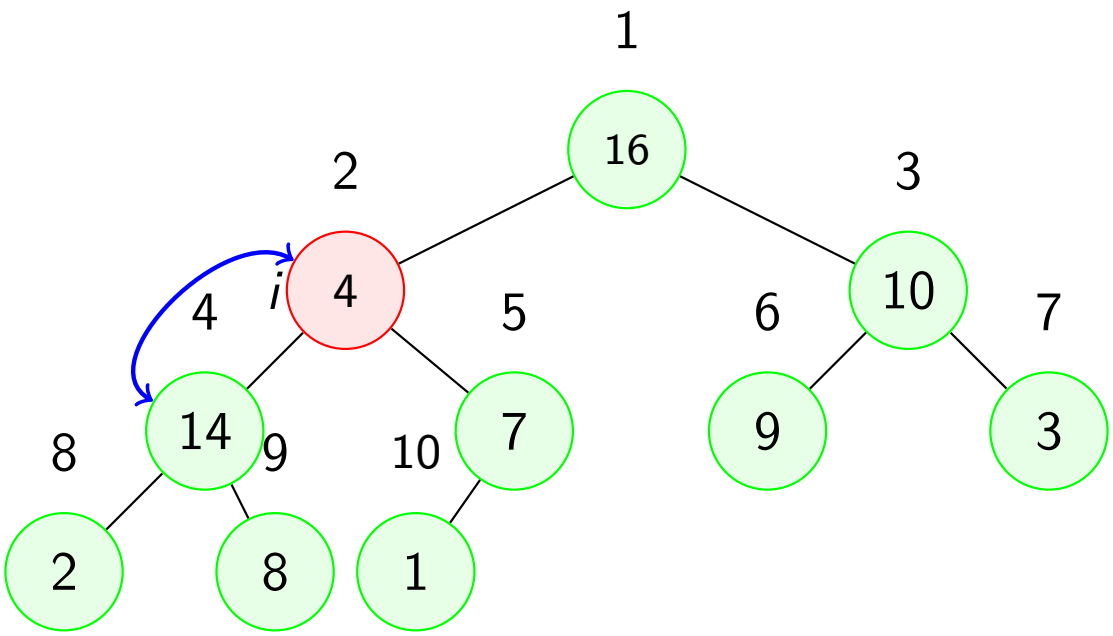
MAX-HEAPIFY(A, i)

```
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

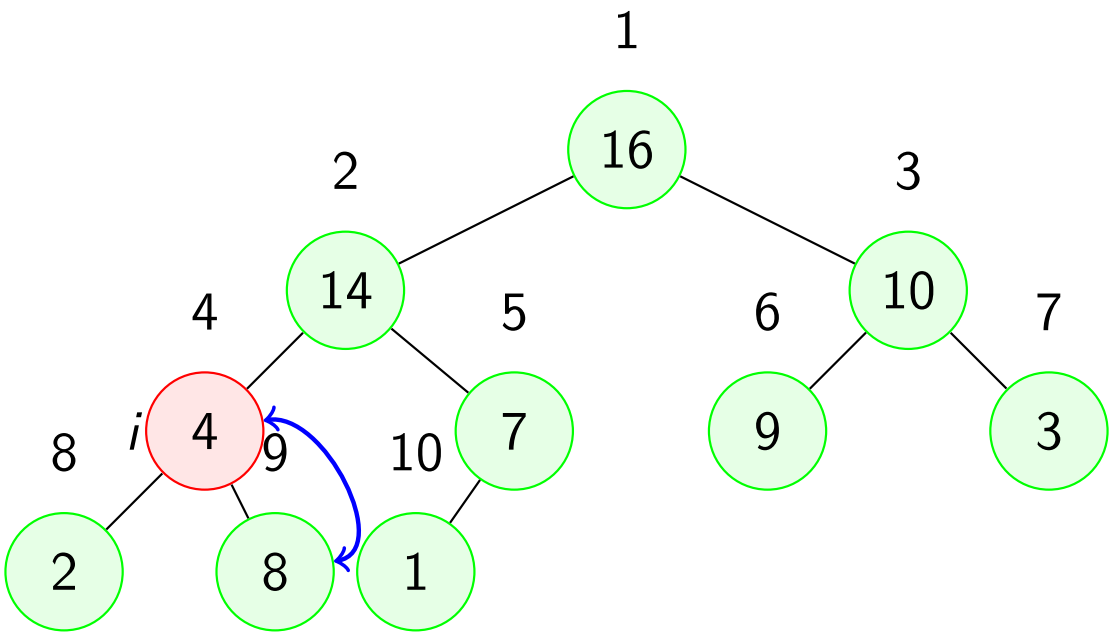
Max-heapify (A, 2)



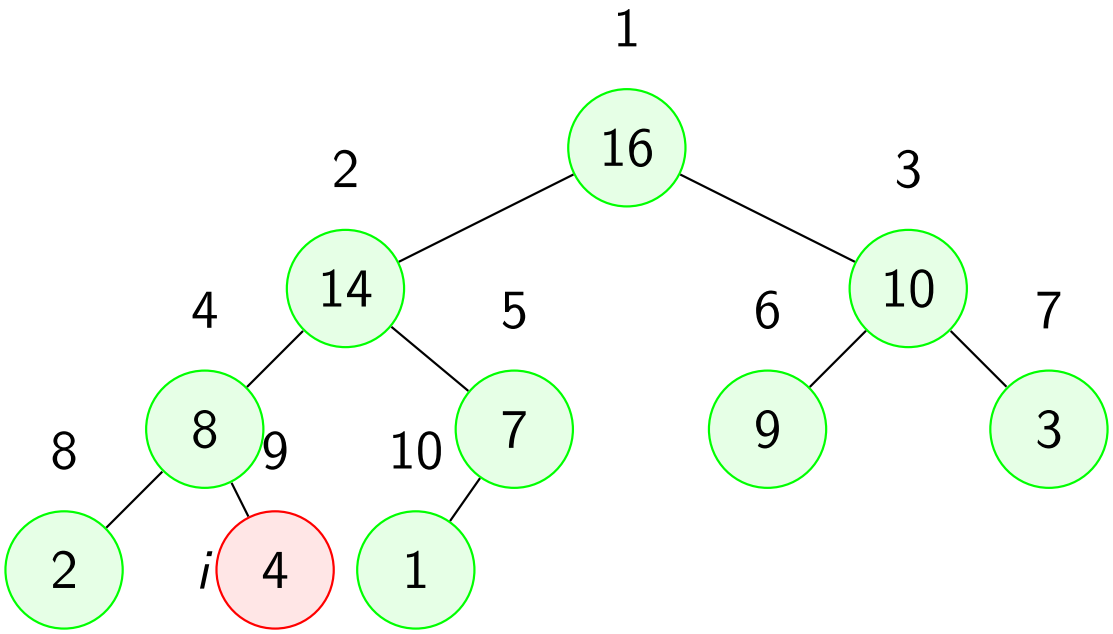
Max-heapify (A, 2)



Max-heapify (A, 4)



Max-heapify (A, 9)



COMPLETE !!!

Max-heapify (Time complexity)

MAX-HEAPIFY(A, i)

```
l = LEFT(i)
r = RIGHT(i)
if l ≤ A.heap-size and A[l] > A[i]
    largest = l
else largest = i
if r ≤ A.heap-size and A[r] > A[largest]
    largest = r
if largest ≠ i
    exchange A[i] with A[largest]
    MAX-HEAPIFY(A, largest)
```

Time complexity

- For a node of height h, the running of MAX-HEAPIFY is $\mathcal{O}(h)$
- For a node of height $logn$ (root of a subtree with n elements), the running of MAX-HEAPIFY is $\mathcal{O}(logn)$

Max-heapify (implementation)

C code

```
void heapify(int arr[], int n, int i)
{
  int largest = i; // Initialize largest as root
  int l = 2 * i + 1; // left child
  int r = 2 * i + 2; // right child

  // Left child is larger than root
  if (l < n && arr[l] > arr[largest])
    largest = l;

  // Right child is larger than largest
  if (r < n && arr[r] > arr[largest])
    largest = r;

  // If largest is not root
  if (largest != i) {
    swap(&arr[i],&arr[largest]);

  // Heapify recursively the affected sub-tree
    heapify(arr, n, largest);
  }
}
```

Build-max-heap

Creating a Max Heap

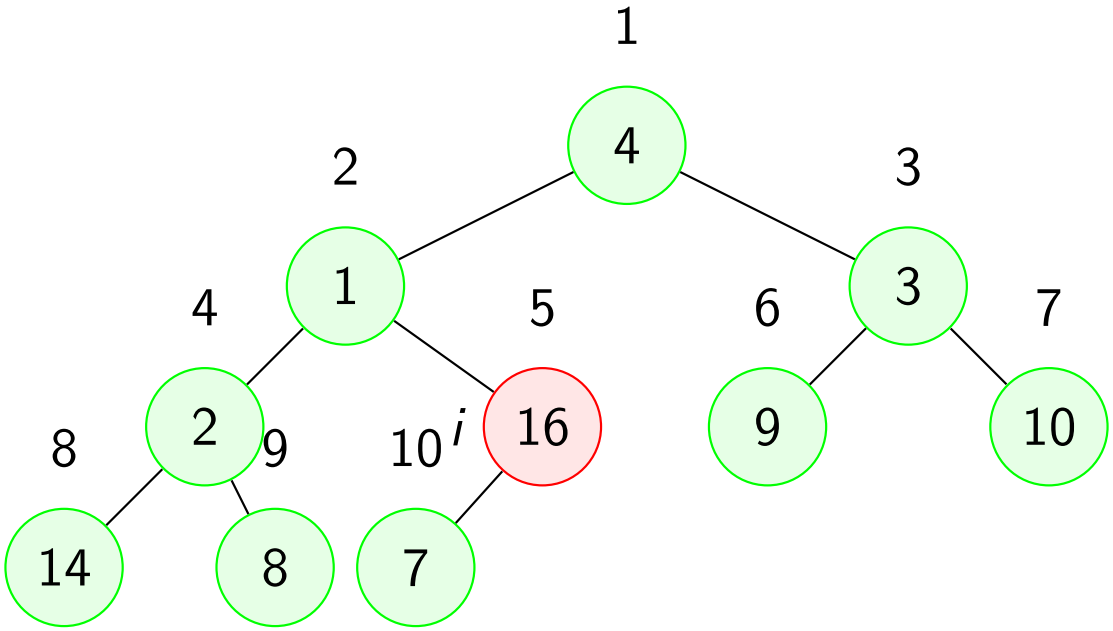
MAX-HEAPIFY can be used in a *bottom-up* manner to convert an array A into a Max Heap

BUILD-MAX-HEAP

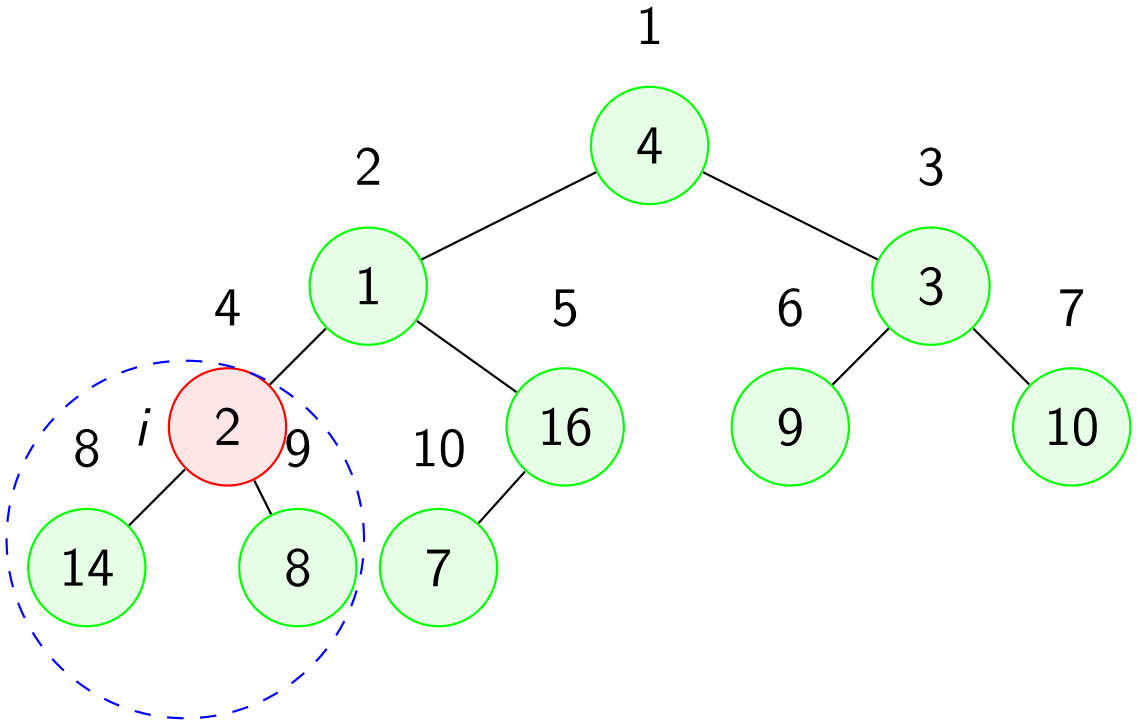
1. A.*heap-size* = A.*length*
2. for $i = \lfloor$ A.*length*$/2 \rfloor$ downto 1
3. MAX-HEAPIFY(A, i)

Build-max-heap (example)

	1	2	3	4	9	5	6	7	8	10
A	4	1	3	2	16	9	10	14	8	7

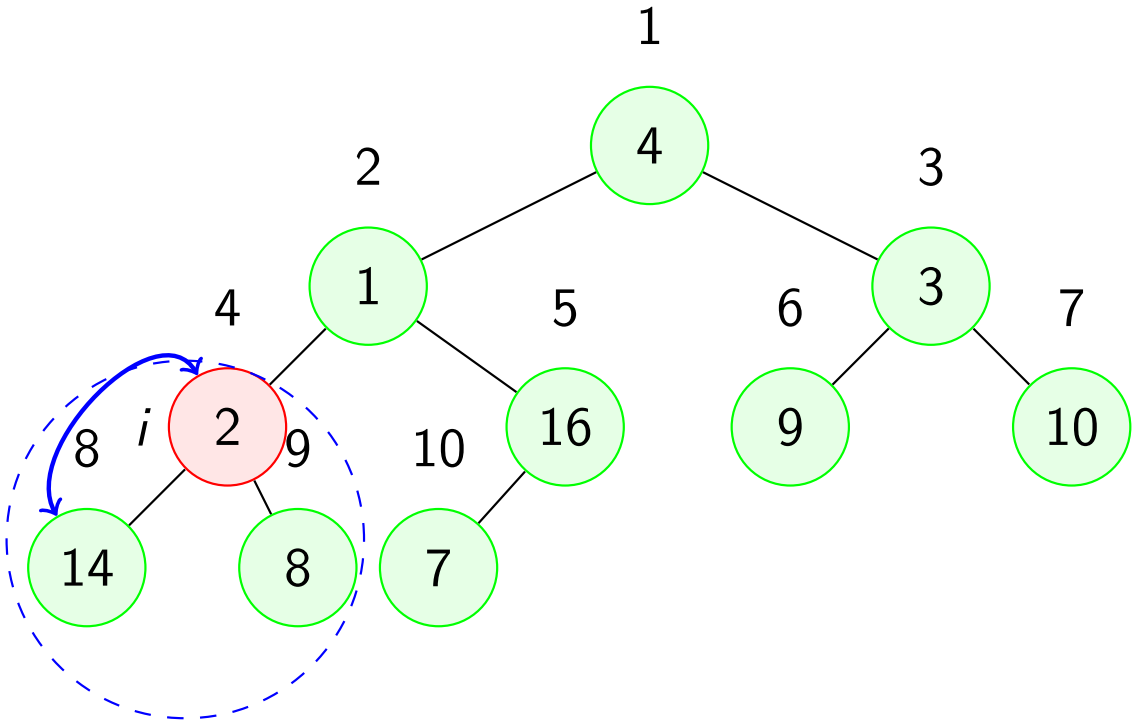


Build-max-heap (example) (contd.)



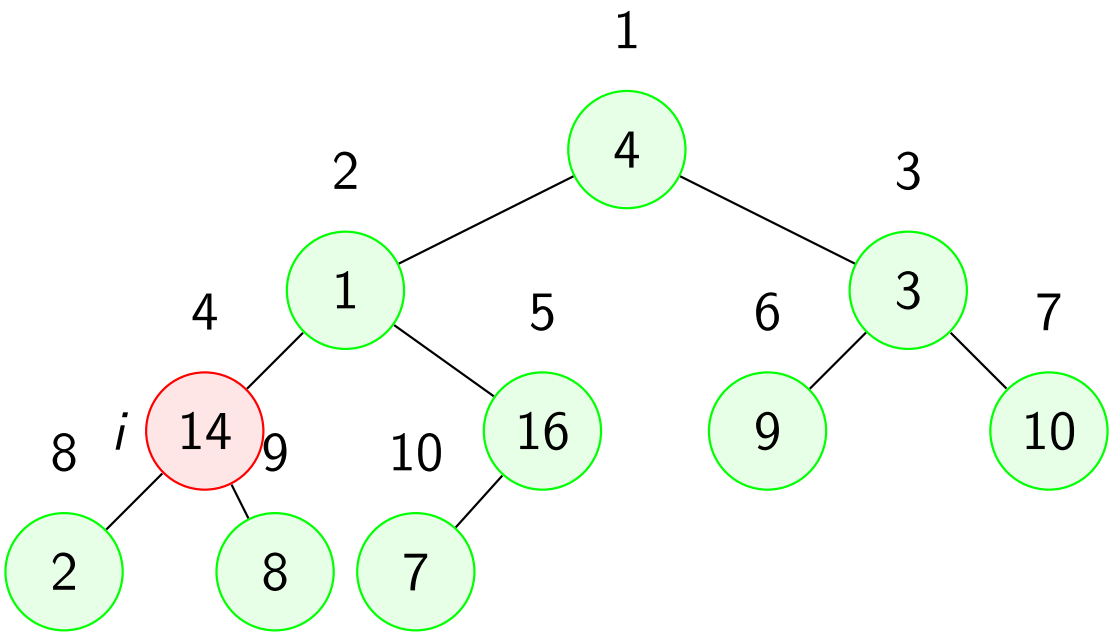
Heap property violated!!

Build-max-heap (example) (contd.)

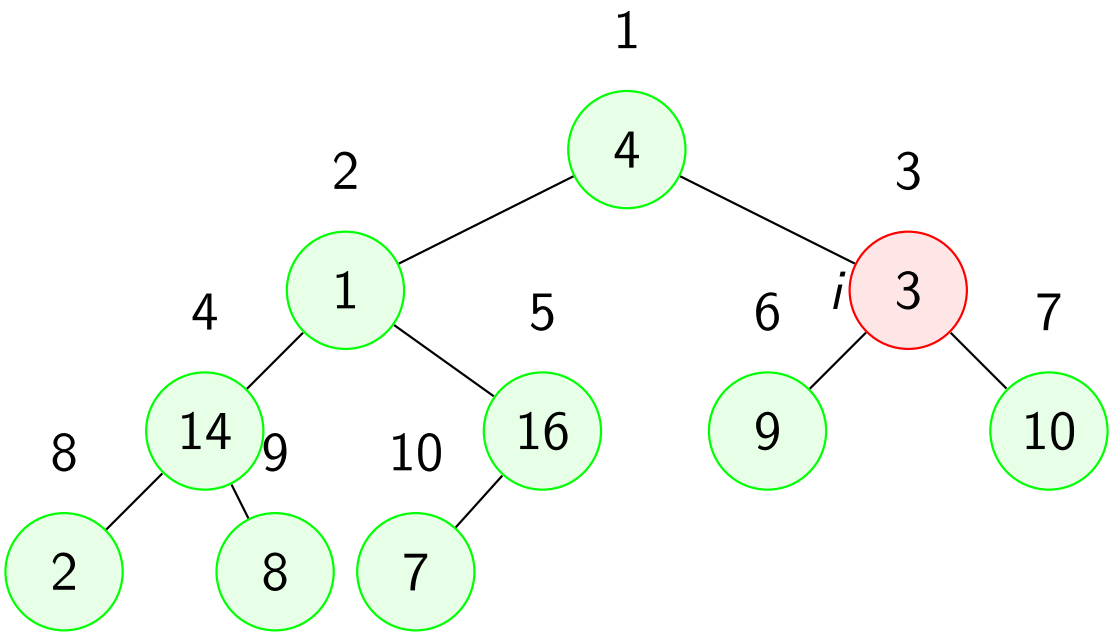


Heap property violated!!

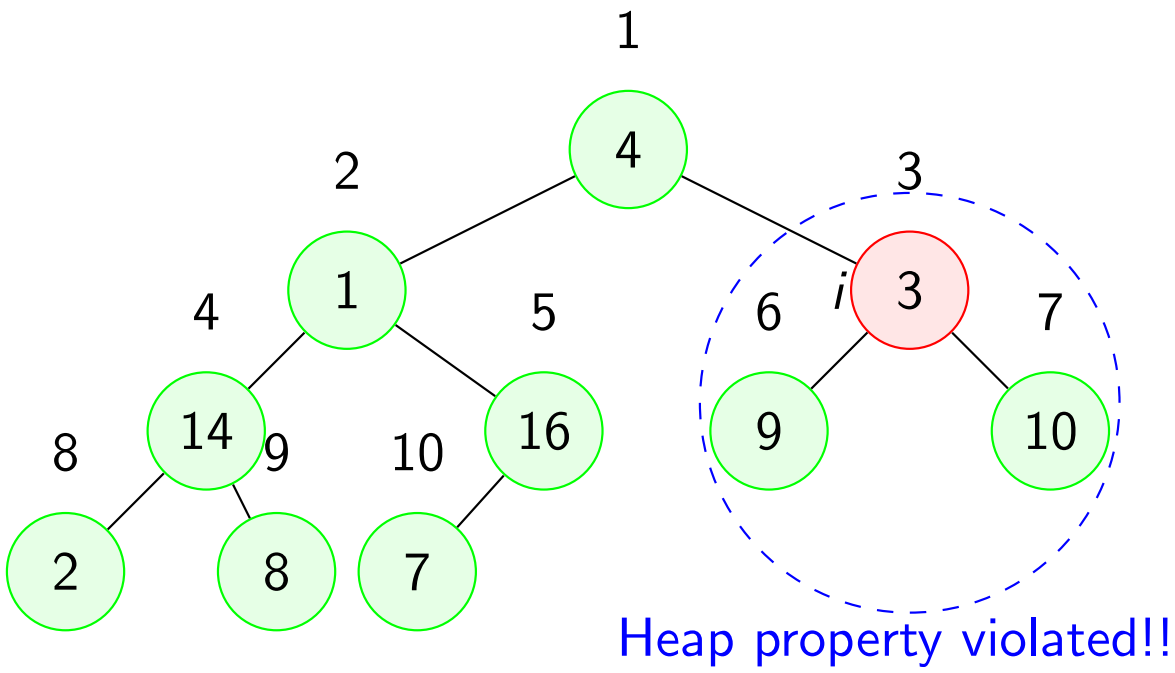
Build-max-heap (example) (contd.)



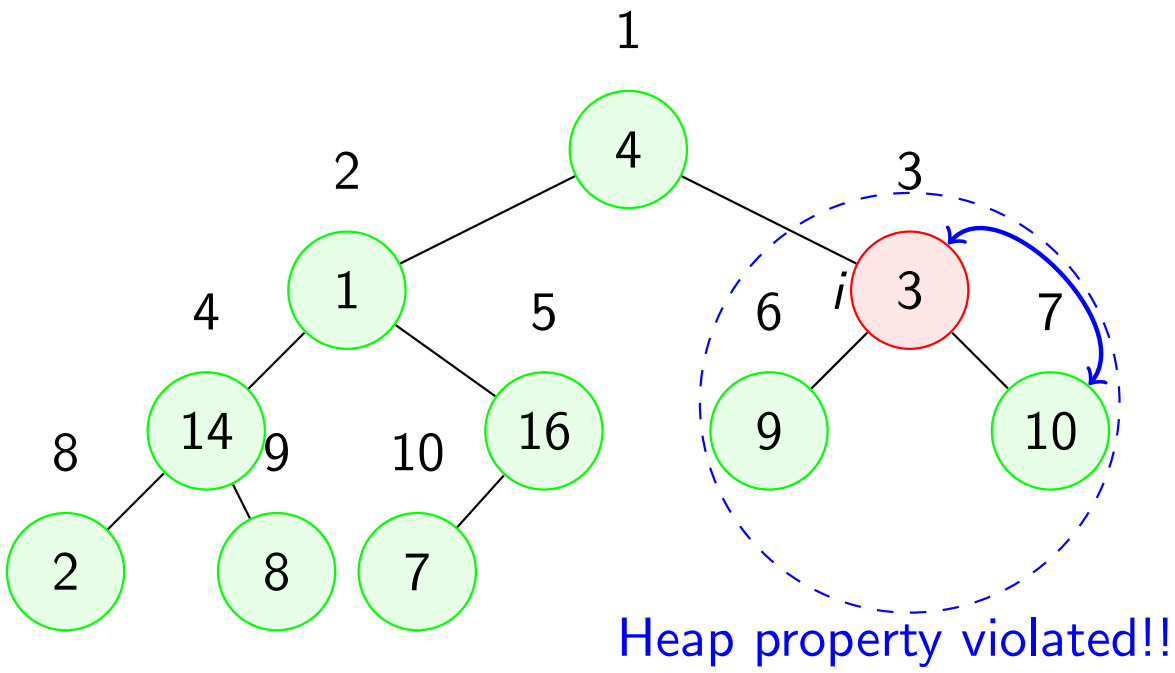
Build-max-heap (example) (contd.)



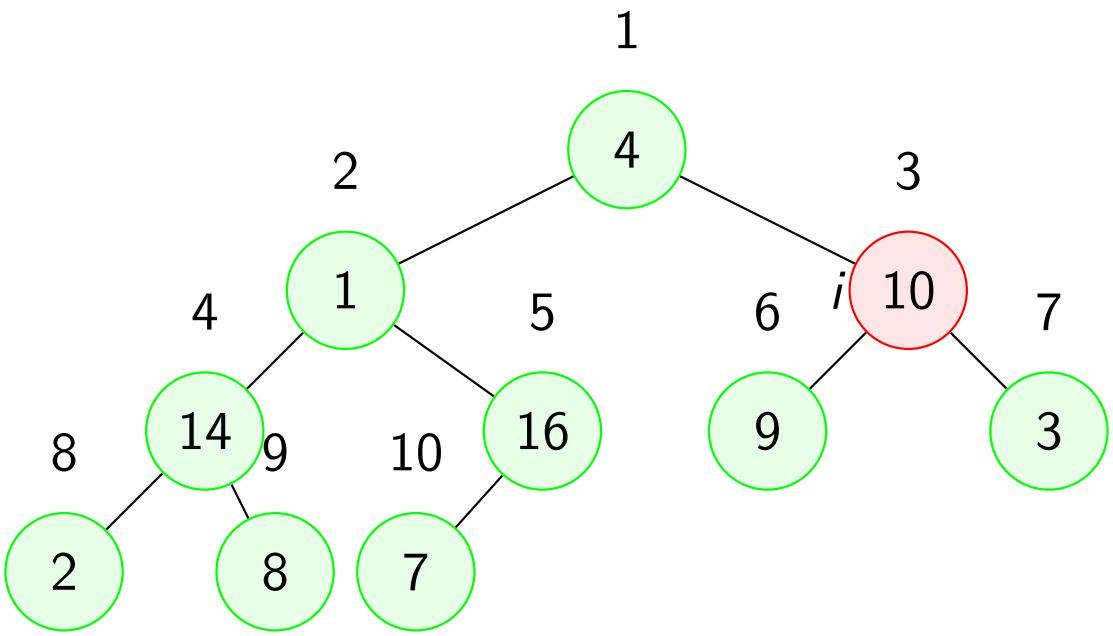
Build-max-heap (example) (contd.)



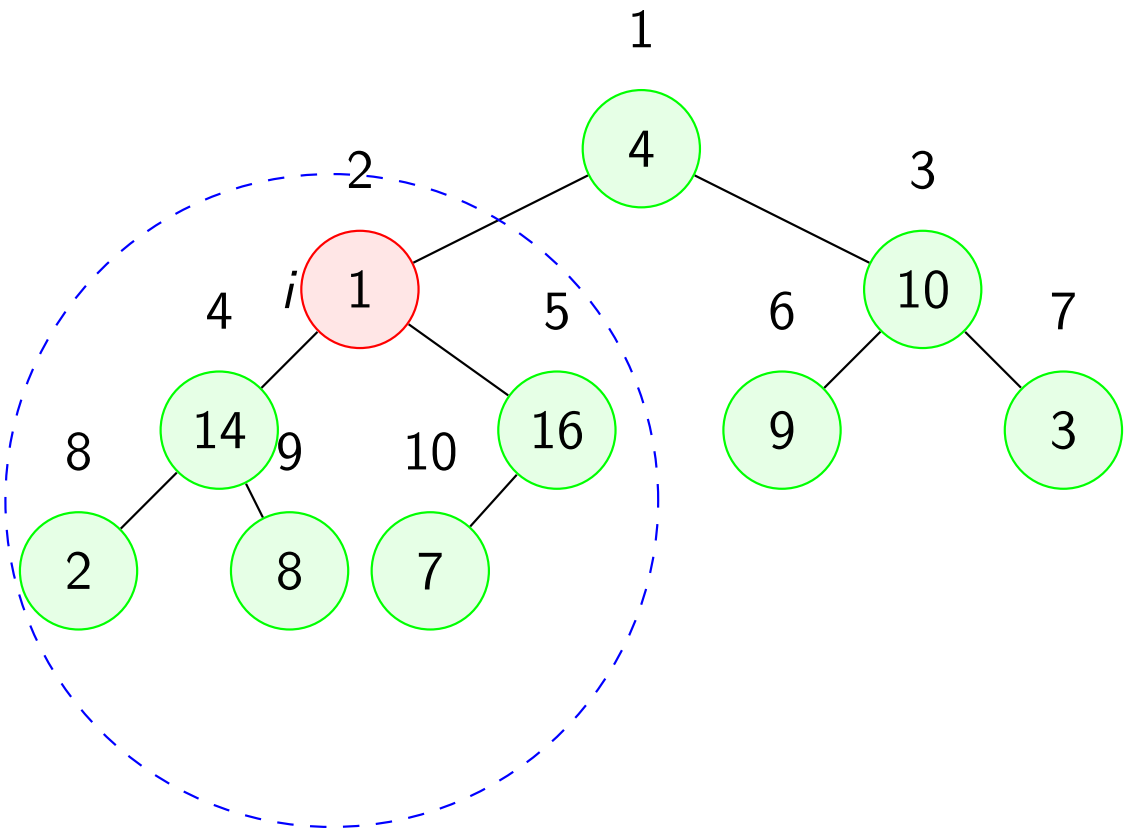
Build-max-heap (example) (contd.)



Build-max-heap (example) (contd.)

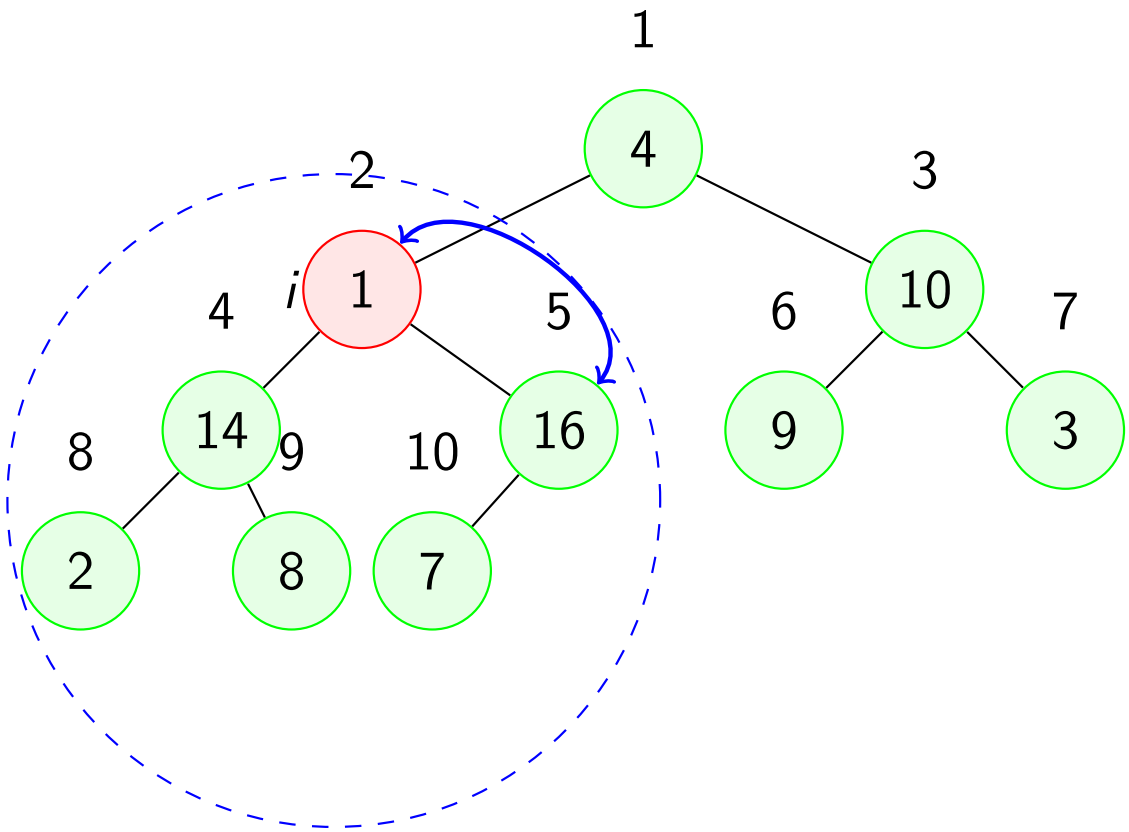


Build-max-heap (example) (contd.)



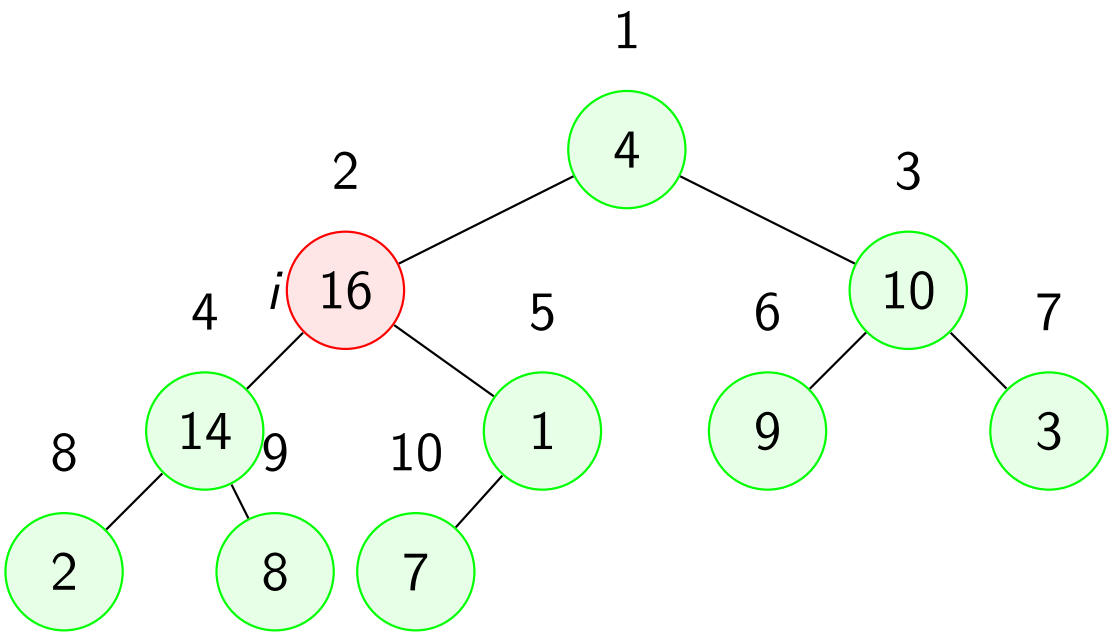
Heap property violated!!

Build-max-heap (example) (contd.)

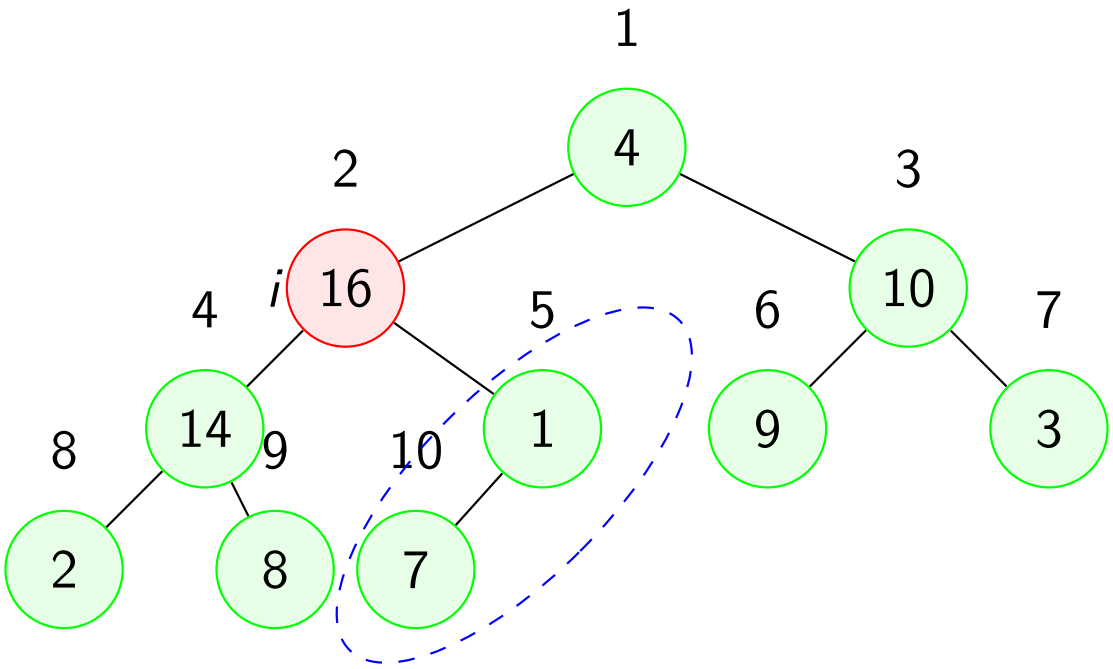


Heap property violated!!

Build-max-heap (example) (contd.)

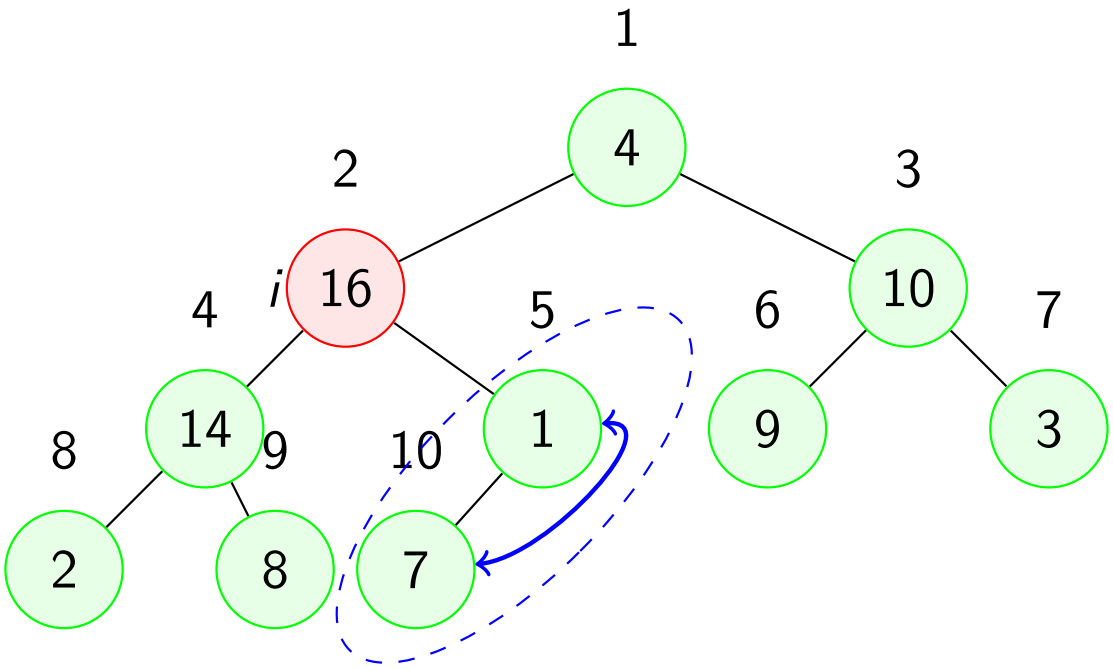


Build-max-heap (example) (contd.)



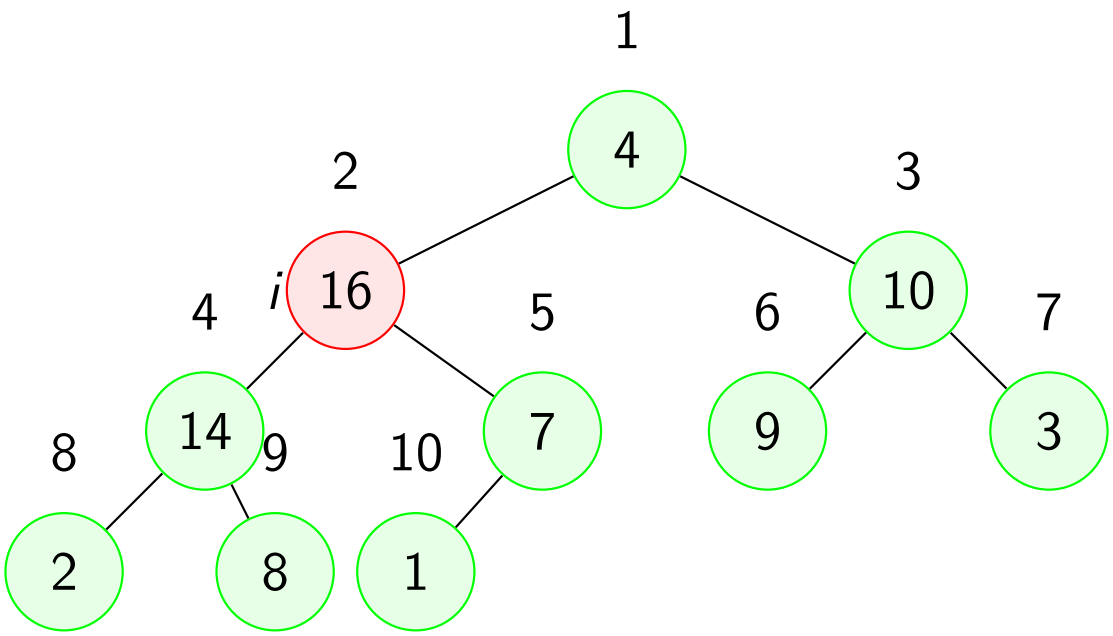
Heap property violated!!

Build-max-heap (example) (contd.)

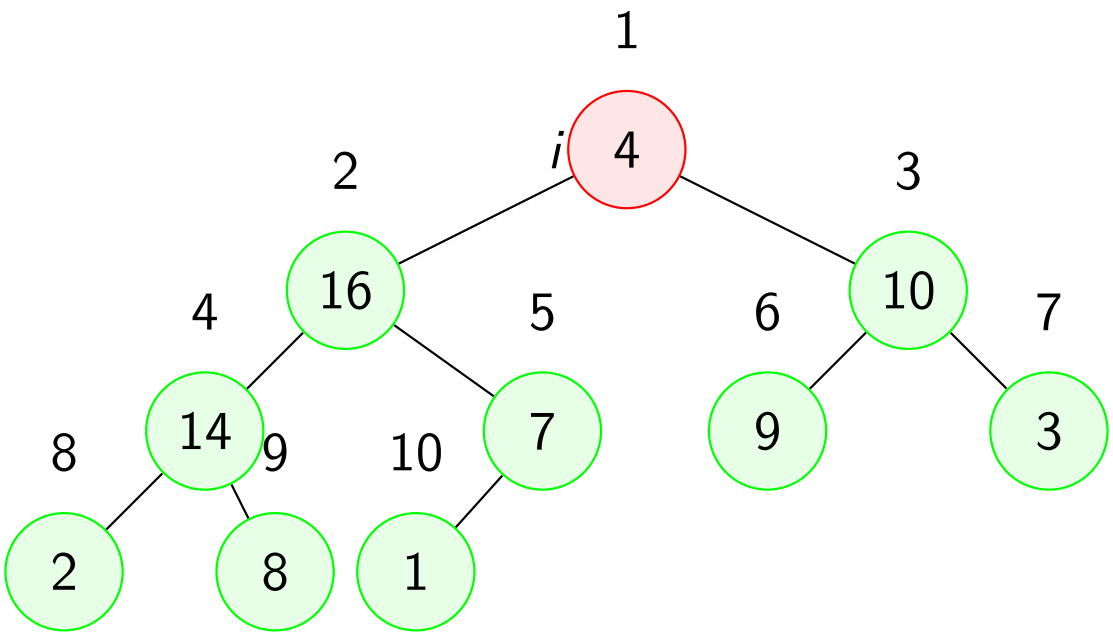


Heap property violated!!

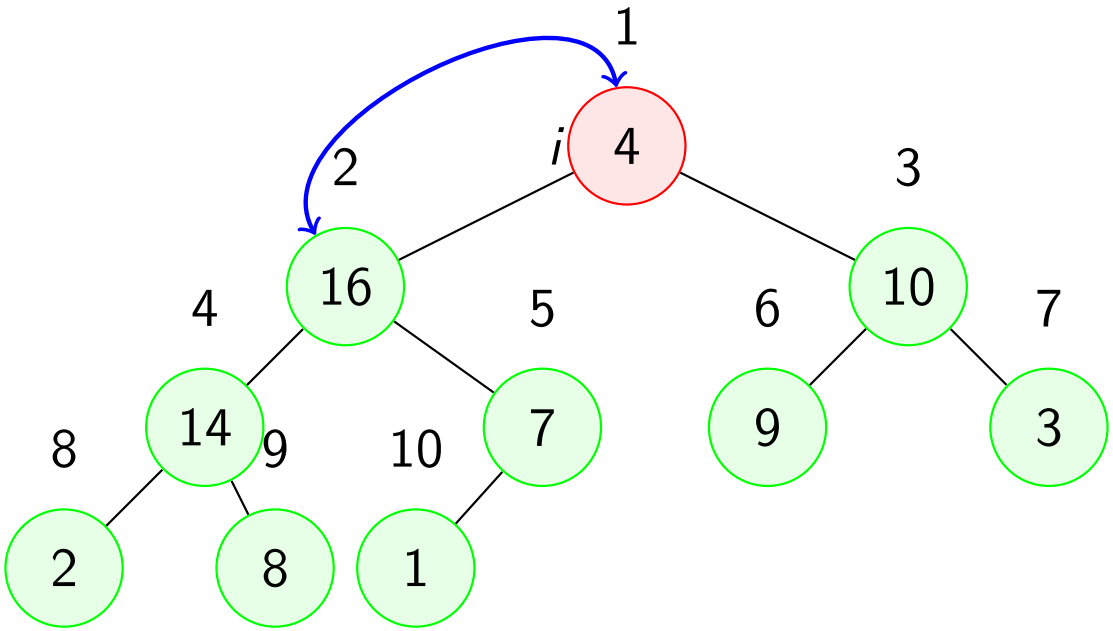
Build-max-heap (example) (contd.)



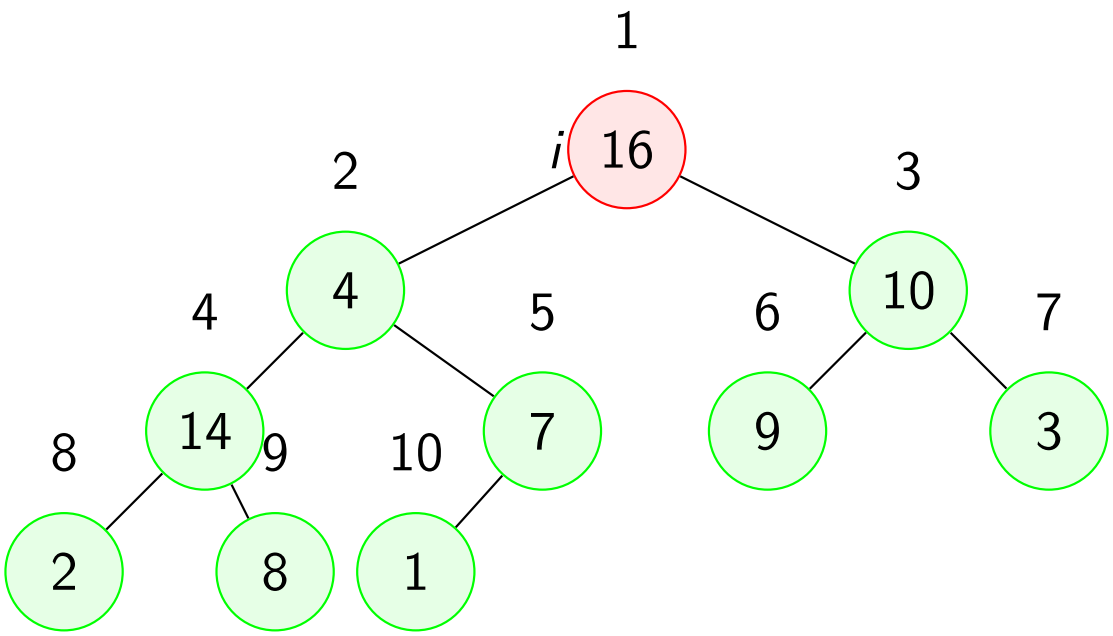
Build-max-heap (example) (contd.)



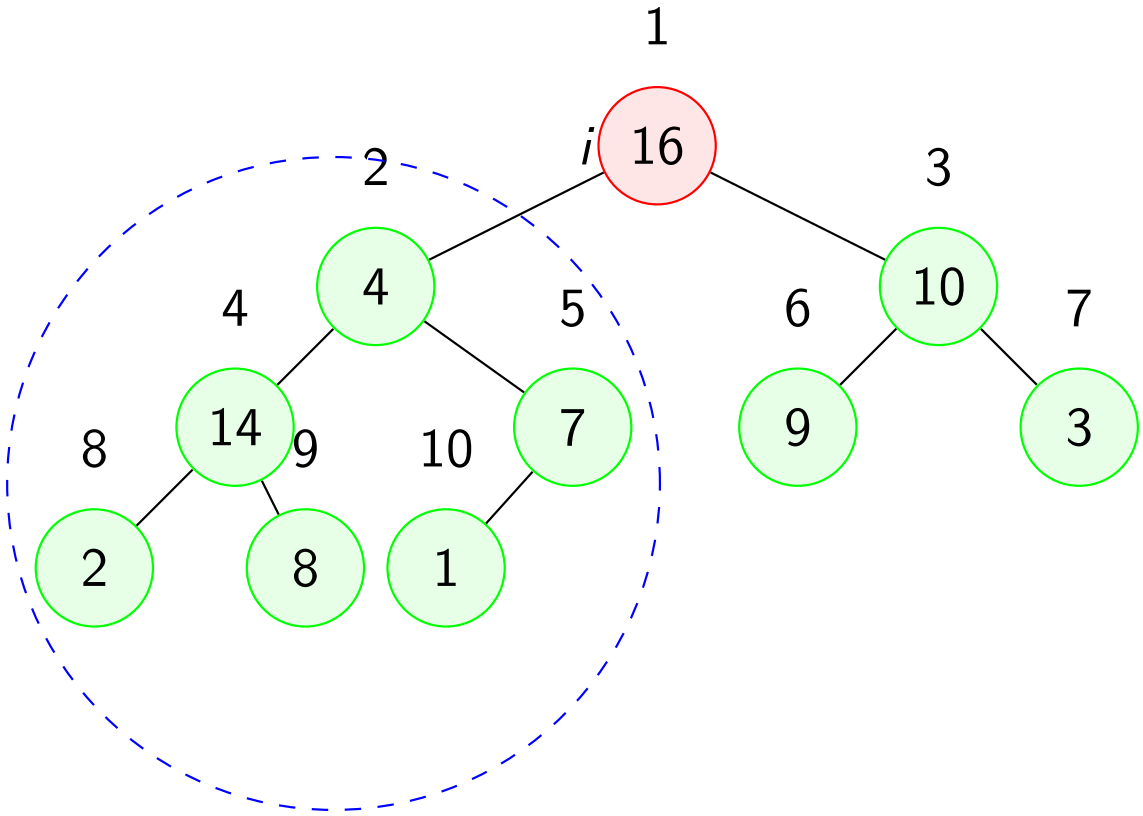
Build-max-heap (example) (contd.)



Build-max-heap (example) (contd.)

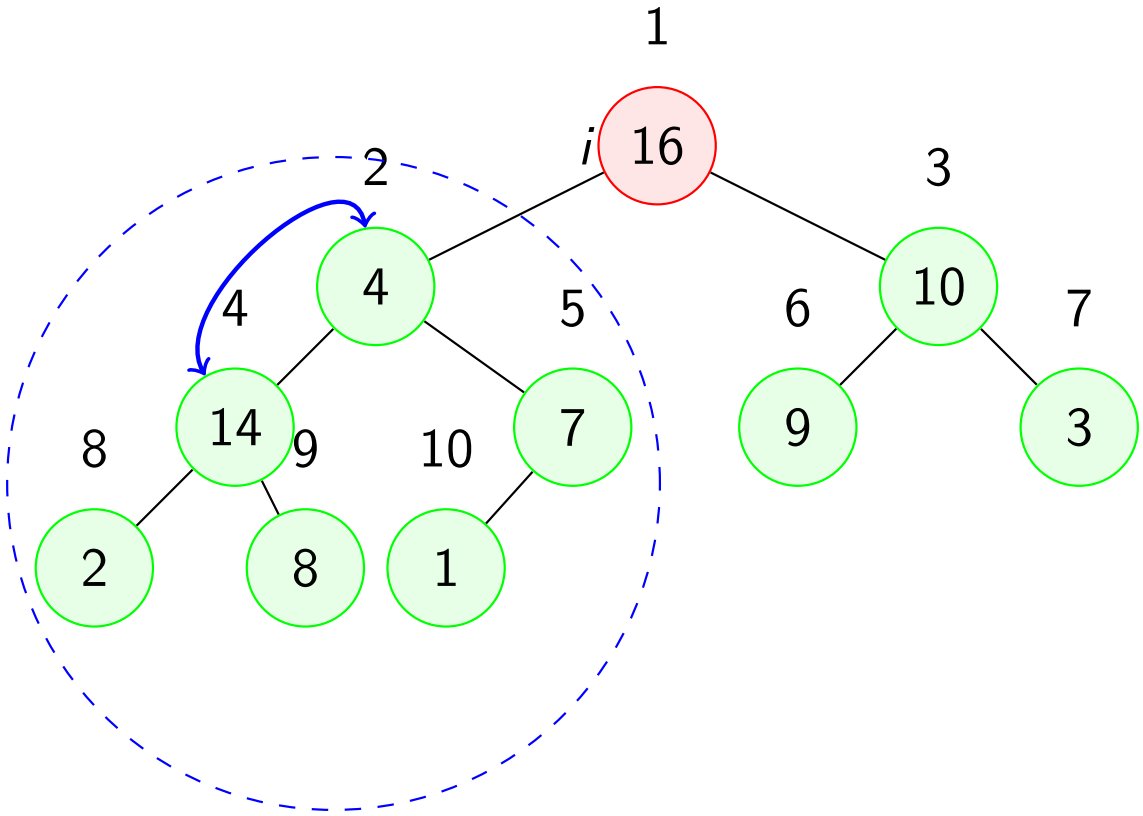


Build-max-heap (example) (contd.)



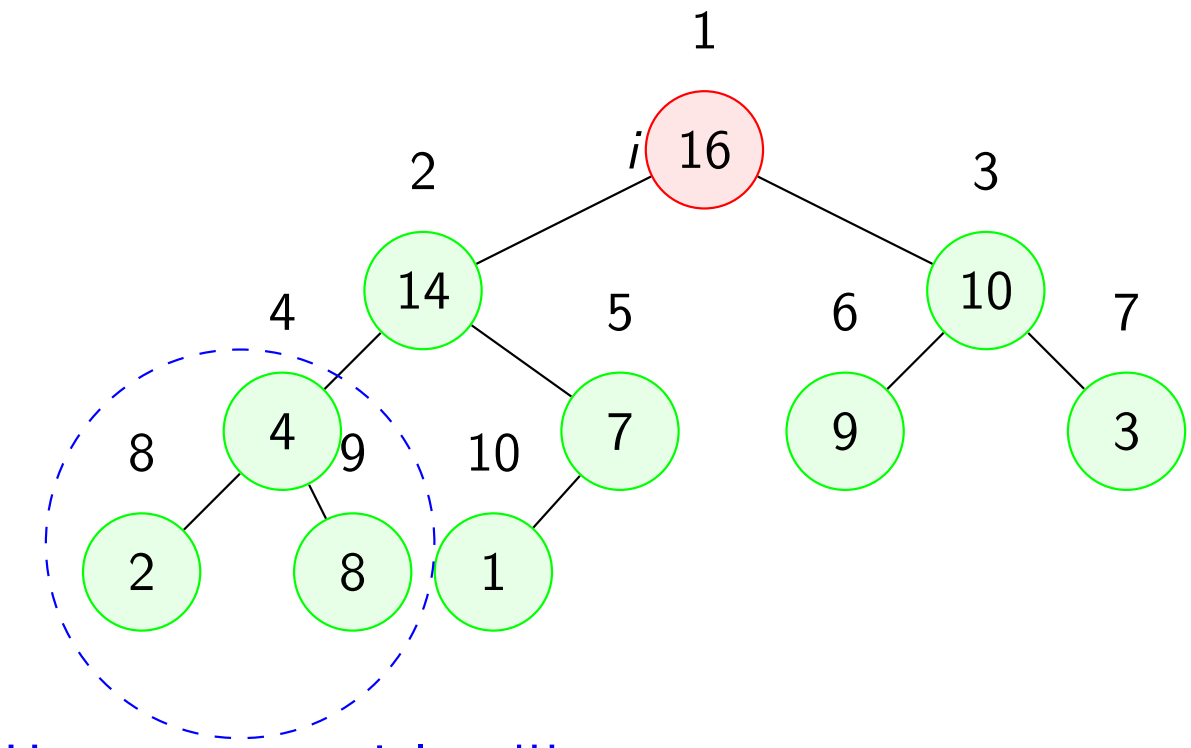
Heap property violated!!

Build-max-heap (example) (contd.)



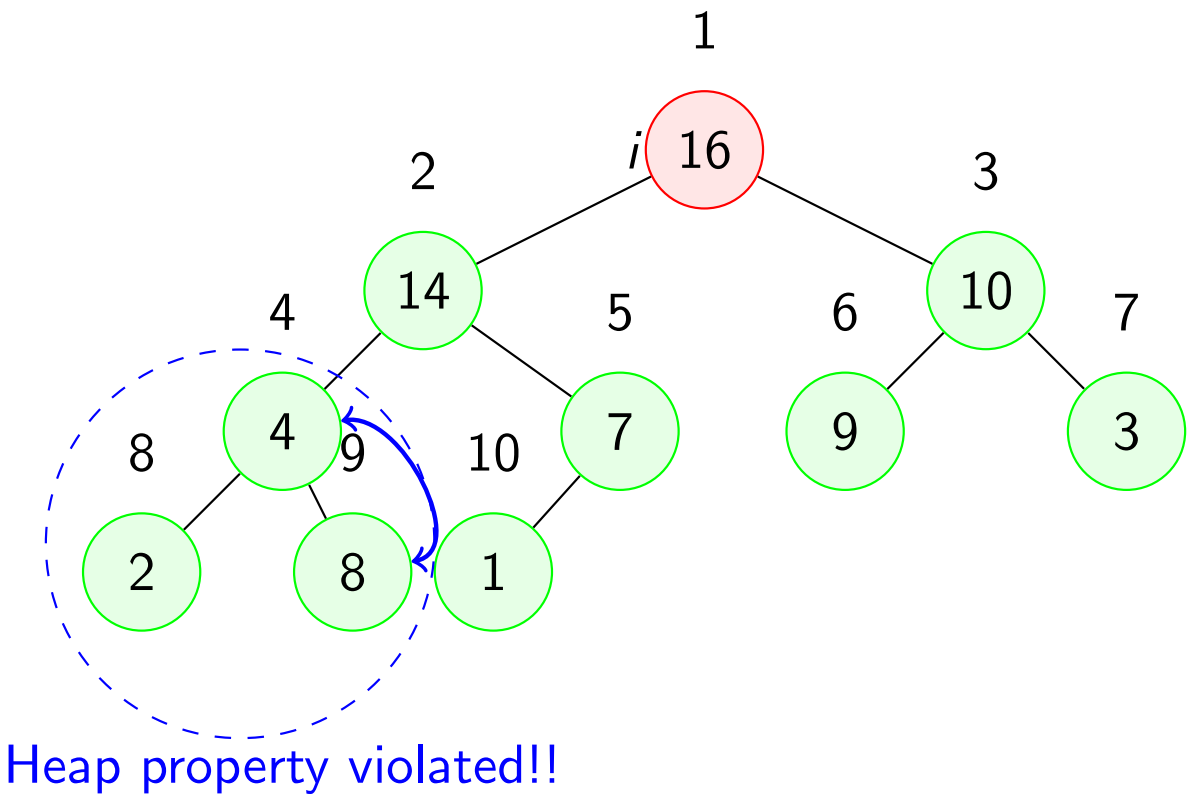
Heap property violated!!

Build-max-heap (example) (contd.)

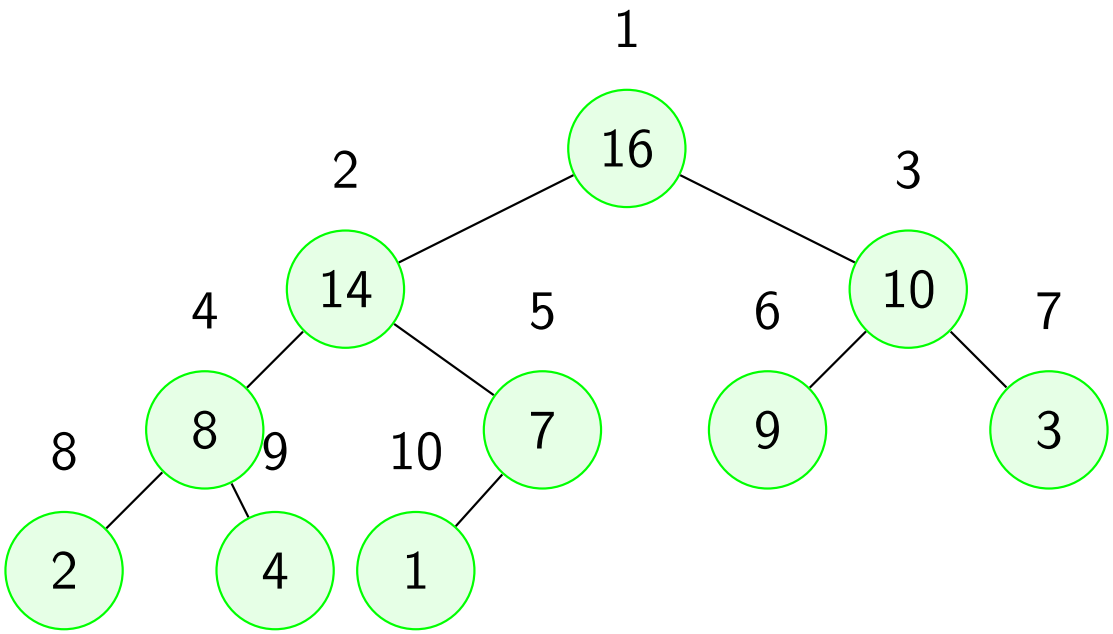


Heap property violated!!

Build-max-heap (example) (contd.)



Build-max-heap (example) (contd.)



COMPLETE!!

Build-max-heap (Time complexity)

BUILD-MAX-HEAP

1. A.*heap-size* = A.*length*
2. for $i = \lfloor$ A.*length*$/2 \rfloor$ downto 1
3. MAX-HEAPIFY(A, i)

Rough upper bound

- Each call to MAX-HEAPIFY costs $\mathcal{O}(logn)$
- BUILD-MAX-HEAP makes $\mathcal{O}(n)$ such calls

Build-max-heap (Time complexity)

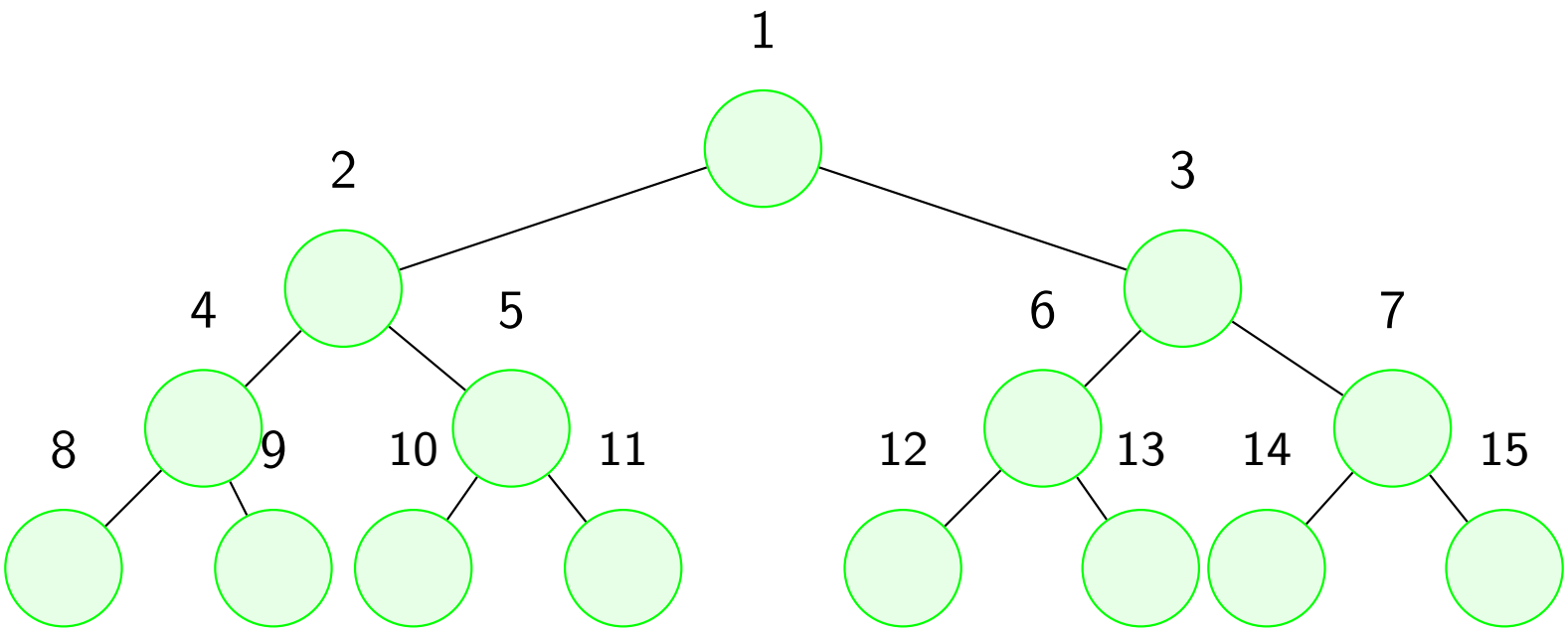
BUILD-MAX-HEAP

1. A.*heap-size* = A.*length*
2. for $i = \lfloor$ A.*length*/2 $\rfloor$ downto 1
3. MAX-HEAPIFY(A, i)

Rough upper bound

- Each call to MAX-HEAPIFY costs $\mathcal{O}(logn)$
- BUILD-MAX-HEAP makes $\mathcal{O}(n)$ such calls
- **Running time of BUILD-MAX-HEAP is** $\mathcal{O}(nlogn)$

Build-max-heap (Time complexity contd.)



Build-max-heap (Time complexity contd.)

Tight upper bound

- n-element heap has height $\lfloor logn \rfloor$
- n-element heap has at most $\lceil \frac{n}{2^{h+1}} \rceil$ nodes of any height h
  - For height $H = 3$, maximum number of nodes $= 2^{H+1}$ - 1 $= 15$
  - If we consider the lowest level and assume a heap rooted at the lowest level, number of nodes $= \lceil \frac{15}{2^{0+1}} \rceil = 8$
  - Similarly for the second lowest, third lowest and root levels, the number of nodes are $\lceil \frac{15}{2^{1+1}} \rceil = 4$, $\lceil \frac{15}{2^{2+1}} \rceil = 2$ and $\lceil \frac{15}{2^{3+1}} \rceil = 1$ respectively
- Running time is $\sum_{h=0}^{\lfloor logn \rfloor} \lceil \frac{n}{2^{h+1}} \rceil \mathcal{O}(h)$

$$
\begin{aligned}
&= \mathcal{O}\left(n \sum_{h=0}^{\lfloor logn \rfloor} \frac{h}{2^{h+1}}\right) \\
&= \mathcal{O}\left(n \sum_{h=0}^{\lfloor logn \rfloor} \frac{h}{2^{h}}\right) \\
&= \mathcal{O}\left(n \sum_{h=0}^{\infty} \frac{h}{2^{h}}\right) \\
&= \mathcal{O}\left(n * 2\right) \\
&= \mathcal{O}\left(n\right)
\end{aligned}
$$

Useful result

- Sum of an infinite geometric series is $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$, $|x| < 1$
- Differentiating and multiplying by x we get, $\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$
- Putting x $= 1/2$, we get $\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1/2)^2} = 2$

HEAPSORT

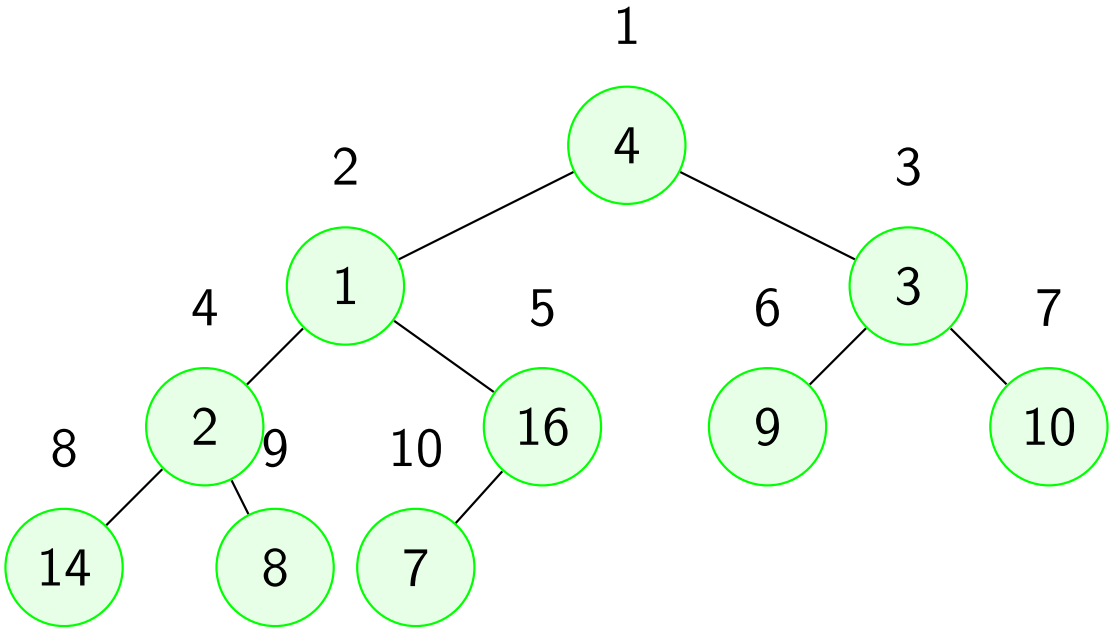
Heapsort

HEAPSORT(A)

```
BUILD-MAX-HEAP(A)
for i = A.length downto 2
    exchange A[1] with A[i]
    A.heap-size = A.heap-size - 1
    MAX-HEAPIFY(A, 1)
```

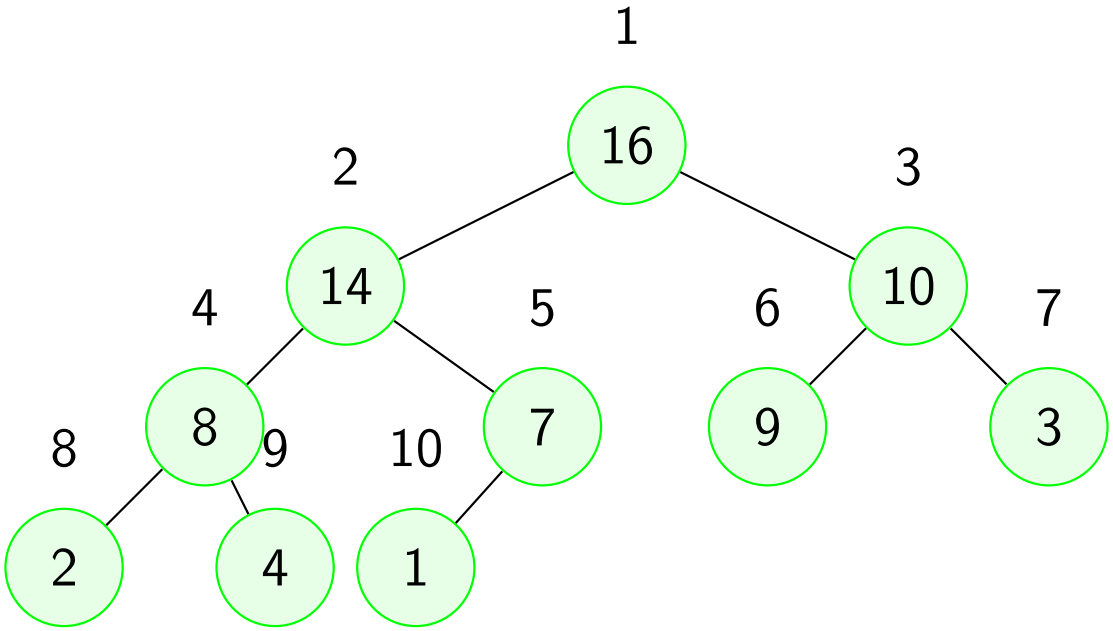
Input array A

	1	2	3	4	5	6	7	8	9	10
A	4	1	3	2	16	9	10	14	8	7



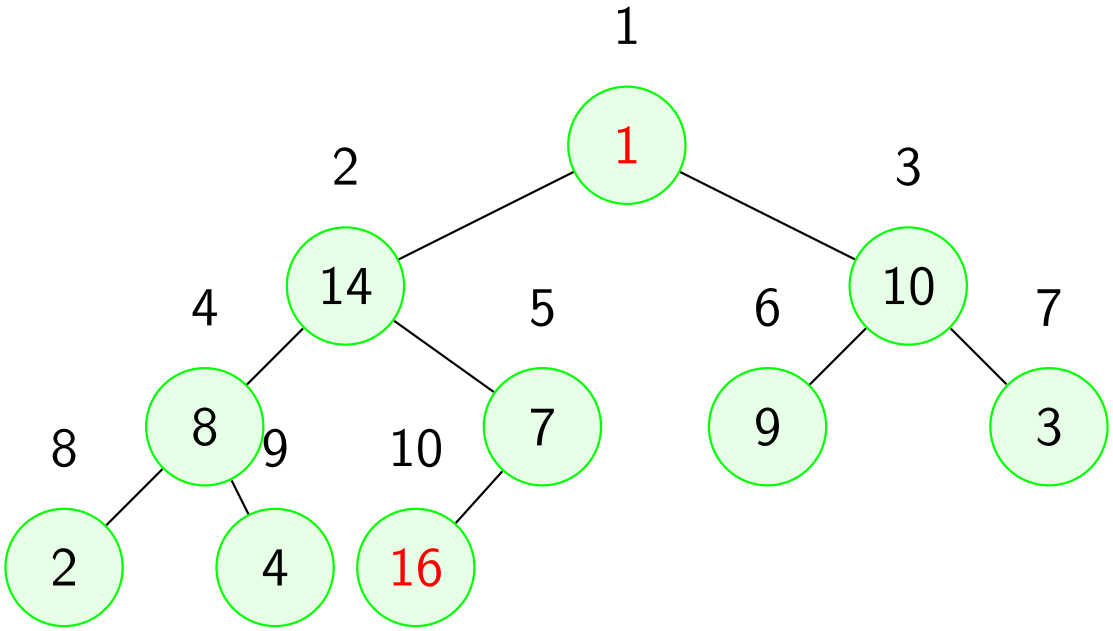
Heapsort: BUILD-MAX-HEAP(A)

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	7	9	3	2	4	1



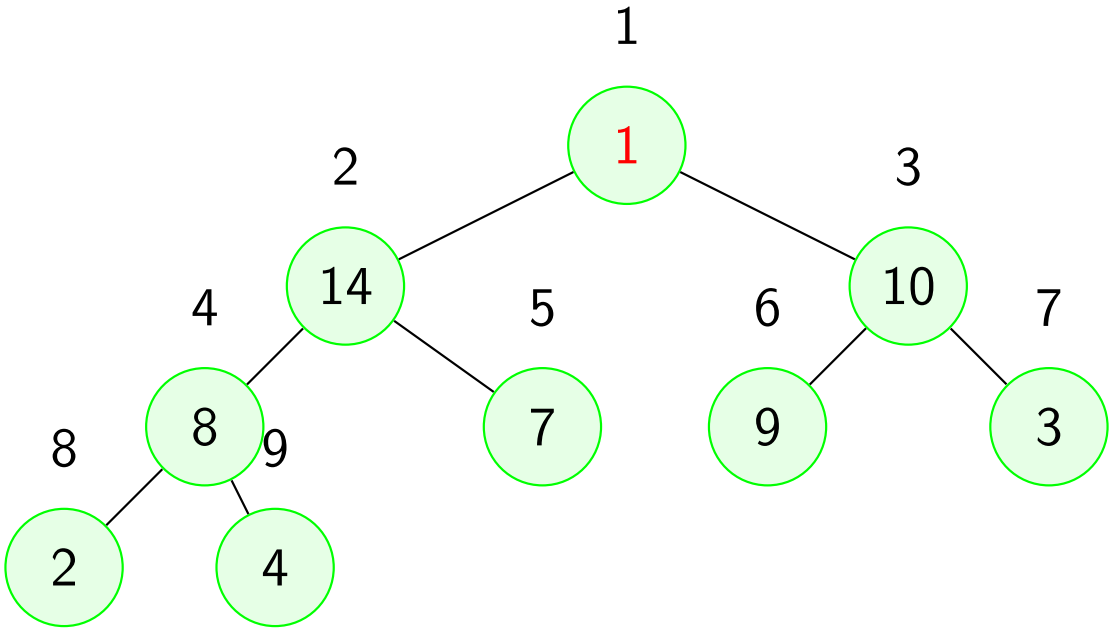
Heapsort: exchange A[1] with A[i]

	1	2	3	4	5	6	7	8	9	10
A	1	14	10	8	7	9	3	2	4	16



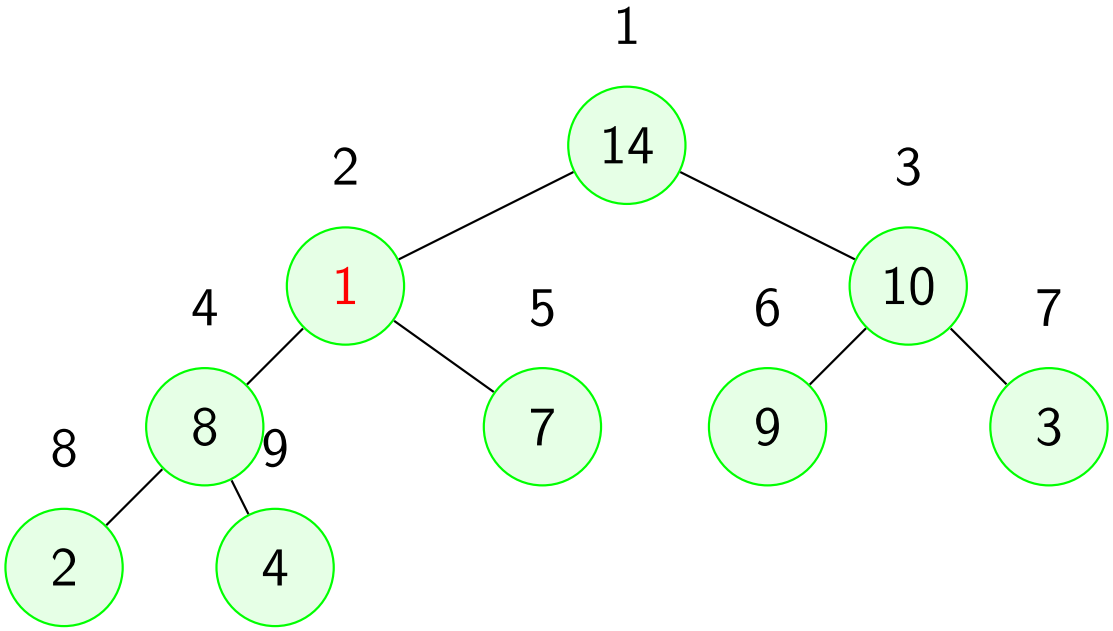
Heapsort: reduce heap size by 1

	1	2	3	4	5	6	7	8	9	
A	1	14	10	8	7	9	3	2	4	16



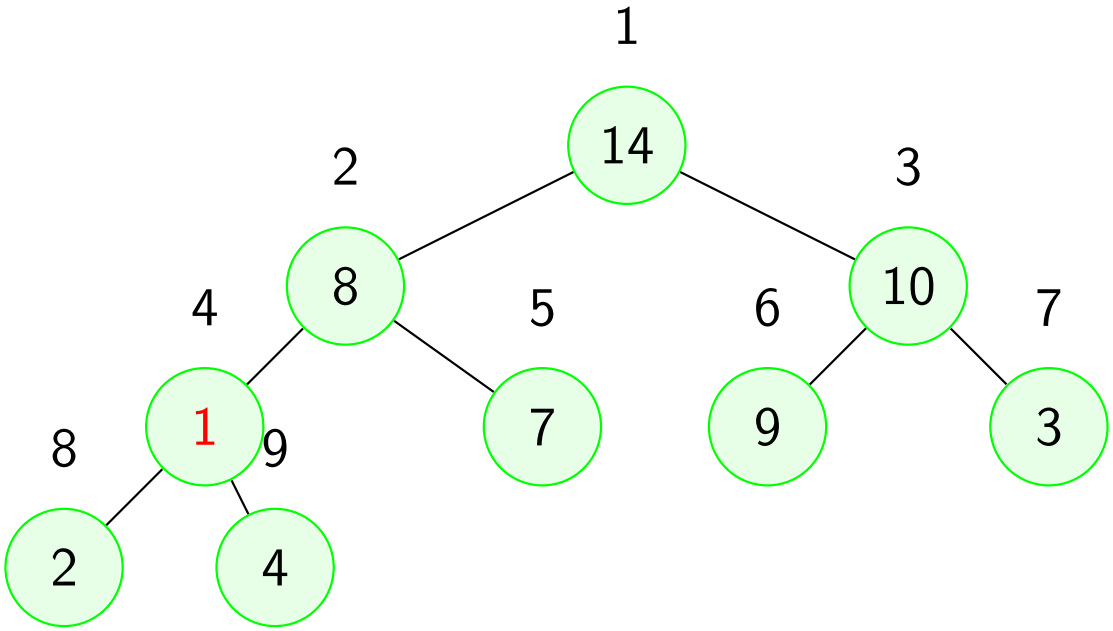
Heapsort: MAX-HEAPIFY(A, 1)

	1	2	3	4	5	6	7	8	9	
A	14	1	10	8	7	9	3	2	4	16



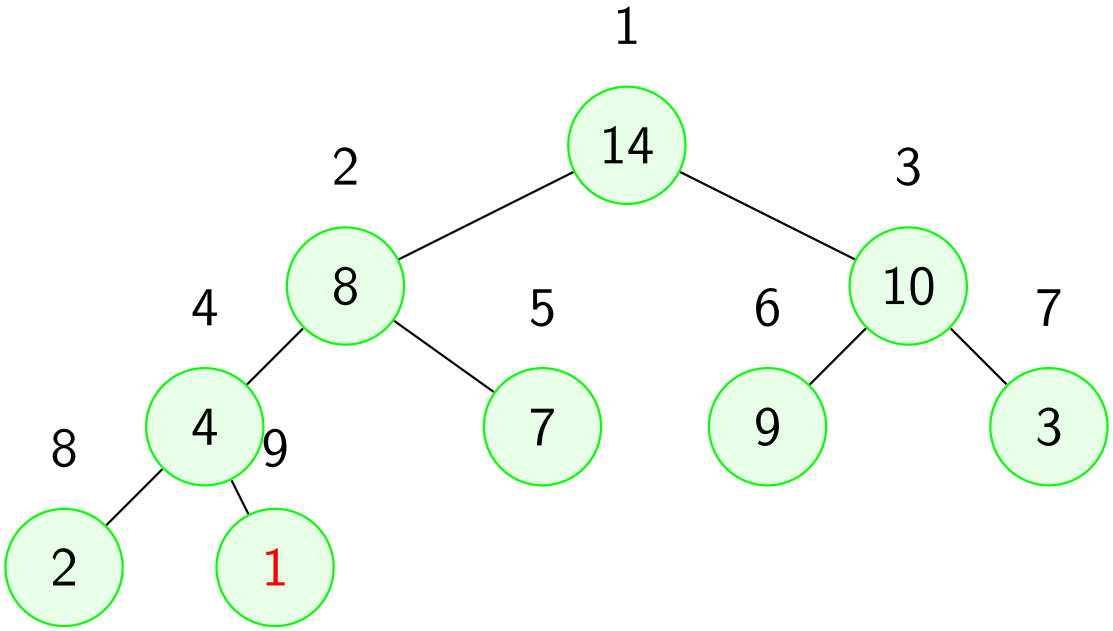
Heapsort: MAX-HEAPIFY(A, 1) (contd.)

	1	2	3	4	5	6	7	8	9	
A	14	8	10	1	7	9	3	2	4	16



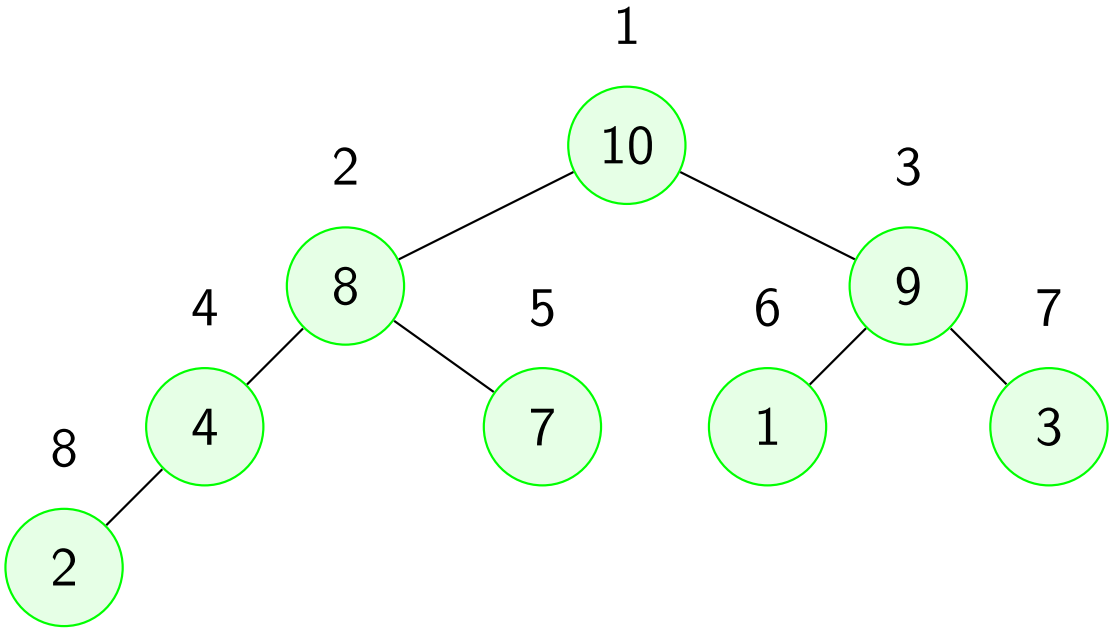
Heapsort: MAX-HEAPIFY(A, 1) (contd.)

	1	2	3	4	5	6	7	8	9	
A	14	8	10	4	7	9	3	2	1	16



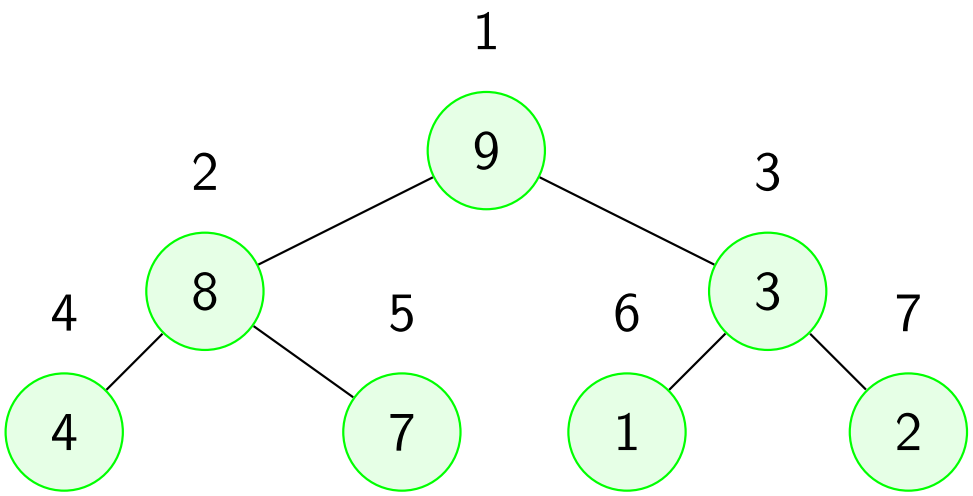
Heapsort (contd.)

	1	2	3	4	5	6	7	8	
A	10	8	9	4	7	1	3	2	14 16



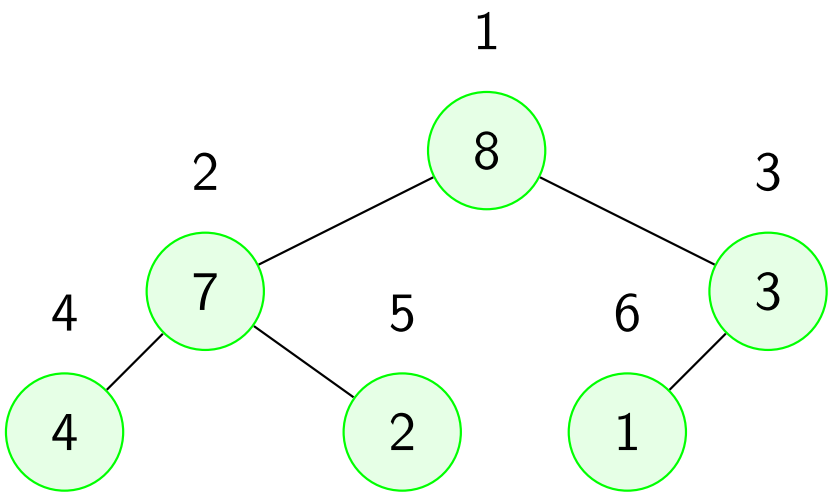
Heapsort (contd.)

	1	2	3	4	5	6	7	
A	9	8	3	4	7	1	2	10 14 16



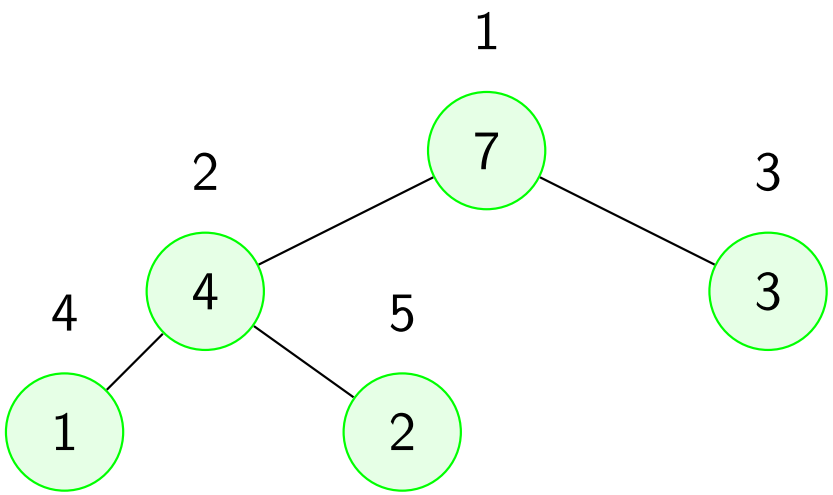
Heapsort (contd.)

	1	2	3	4	5	6						
A	8	7	3	4	2	1	9	10	14	16		



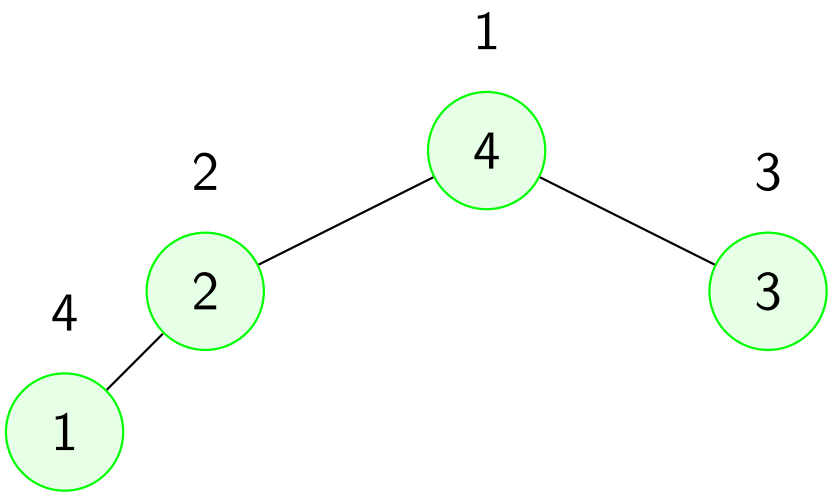
Heapsort (contd.)

	1	2	3	4	5						
A	7	4	3	1	2	8	9	10	14	16	



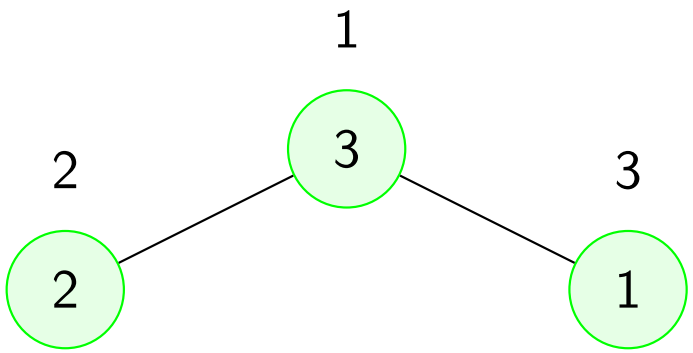
Heapsort (contd.)

	1	2	3	4							
A	4	2	3	1	7	8	9	10	14	16	



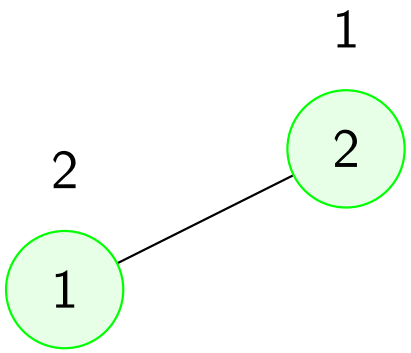
Heapsort (contd.)

	1	2	3							
A	3	2	1	4	7	8	9	10	14	16



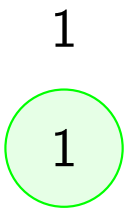
Heapsort (contd.)

	1	2								
A	2	1	3	4	7	8	9	10	14	16



Heapsort (final step)

	1										
A	$\boxed{1}$	2	3	4	7	8	9	10	14	16	



Heapsort (Time complexity)

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. for i = A.*length* downto 2
3. exchange A[1] with A[i]
4. A.*heap-size* = A.*heap-size* - 1
5. MAX-HEAPIFY(A, 1)

- BUILD-MAX-HEAP costs $\mathcal{O}(n)$
- Each call to MAX-HEAPIFY costs $\mathcal{O}(logn)$
- There are n - 1 calls to MAX-HEAPIFY

Heapsort (Time complexity)

HEAPSORT(A)

1. BUILD-MAX-HEAP(A)
2. for i = A.*length* downto 2
3. exchange A[1] with A[i]
4. A.*heap-size* = A.*heap-size* - 1
5. MAX-HEAPIFY(A, 1)

- BUILD-MAX-HEAP costs $\mathcal{O}(n)$
- Each call to MAX-HEAPIFY costs $\mathcal{O}(logn)$
- There are n - 1 calls to MAX-HEAPIFY
- **Running time of HEAPSORT is** $\mathcal{O}(nlogn)$

PRIORITY QUEUE

Priority Queue

Property

- Data structure: Maintaining a set S with a value *key*
- Max- or min-priority queue

Max-priority queue: operations

- **INSERT(S, x)**: Insert x into S ($S = S \cup \{x\}$)
- **MAXIMUM(S)**: Return the largest key in S
- **EXTRACT-MAX(S)**: Remove and return the largest key
- **INCREASE-KEY(S, i, key)**: Increase the key at ith position to the new value *key* if *key* does not have a lower value

Utility

- Max-priority-queue is used for process scheduling to extract the one with the highest priority (using EXTRACT-MAX operation)
- New process added to the queue by INSERT operation

MAXIMUM(S)

HEAP-MAXIMUM(S)

```
return A[1]
```

Running time is $\mathcal{O}(1)$

EXTRACT-MAX(S)

HEAP-EXTRACT-MAX(S)

1. if A.*heap-size* < 1
2. error "heap overflow"
3. $max = A[1]$
4. $A[1] = A[\ A.heap\text{-}size]$
5. A.*heap-size* = A.*heap-size* - 1
6. MAX-HEAPIFY(A,1)
7. return *max*

Running time is $\mathcal{O}(logn)$ (running time for MAX-HEAPIFY)

HEAP-INCREASE-KEY

HEAP-INCREASE-KEY (A, i, key)

```
if key < A[i]
    error "new key smaller than the current key"
A[i] = key
while i > 1 and A[PARENT(i)] < A[i]
    exchange A[PARENT(i)] with A[i]
    i = A[PARENT(i)]
```

- For a heap of n elements, the path from any node to the root has length $\mathcal{O}(logn)$

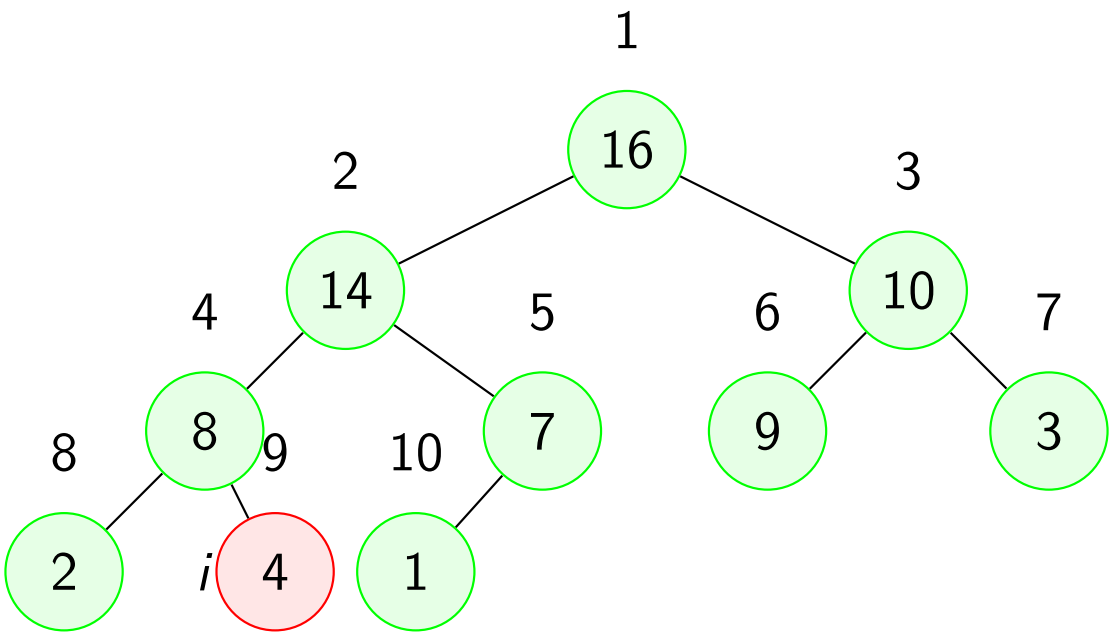
HEAP-INCREASE-KEY

HEAP-INCREASE-KEY (A, i, key)

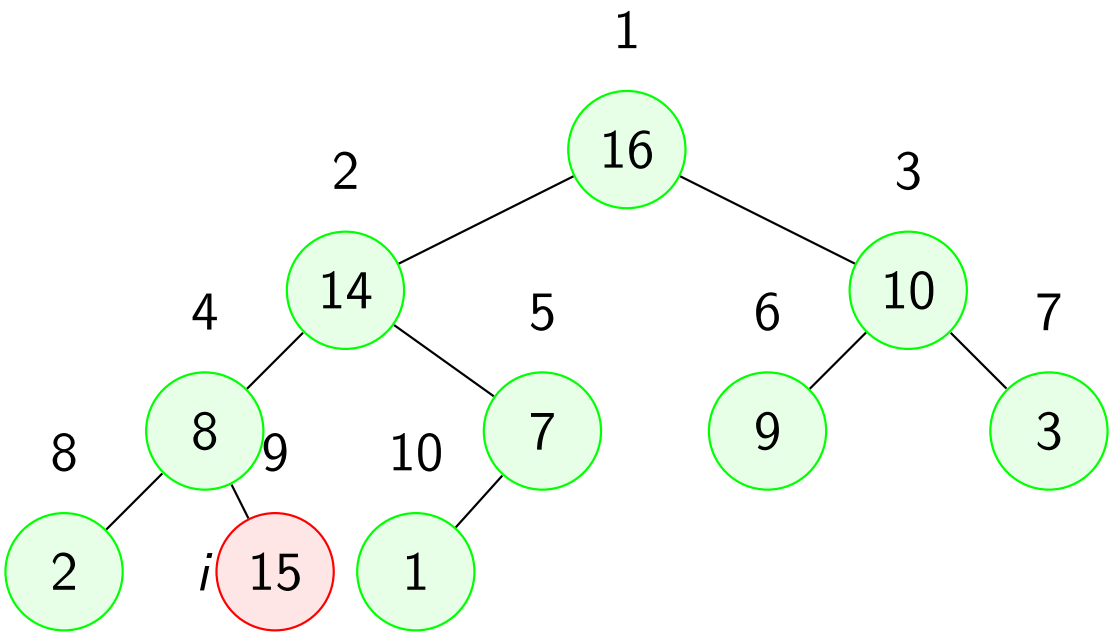
```
if key < A[i]
    error "new key smaller than the current key"
A[i] = key
while i > 1 and A[PARENT(i)] < A[i]
    exchange A[PARENT(i)] with A[i]
    i = A[PARENT(i)]
```

- For a heap of n elements, the path from any node to the root has length $\mathcal{O}(logn)$
- Running time of HEAP-INCREASE-KEY is $\mathcal{O}(logn)$

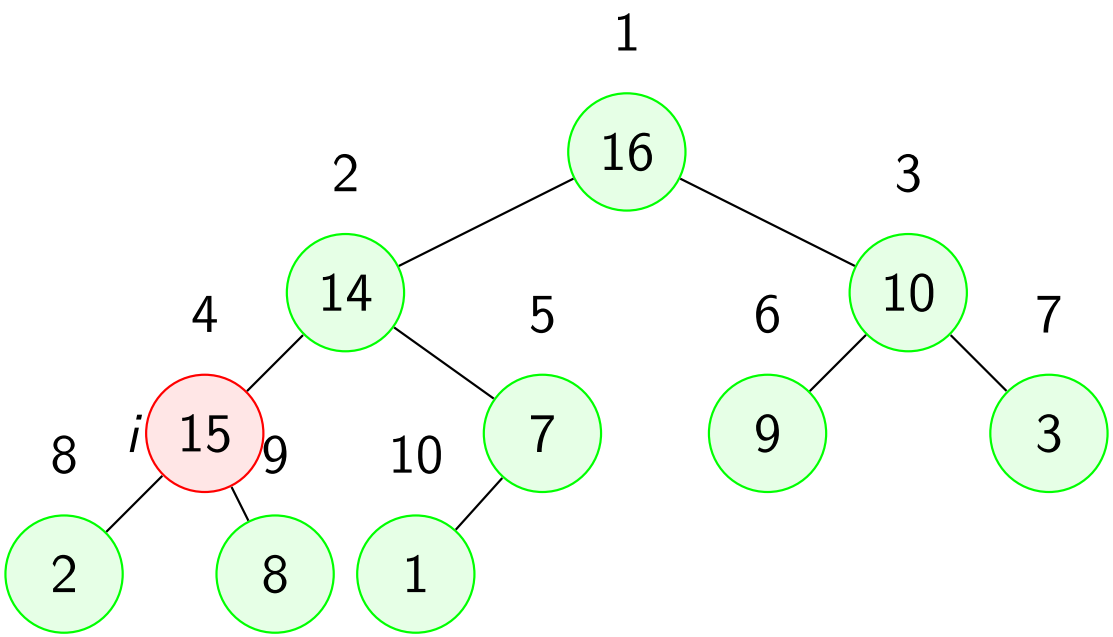
HEAP-INCREASE-KEY: before increase



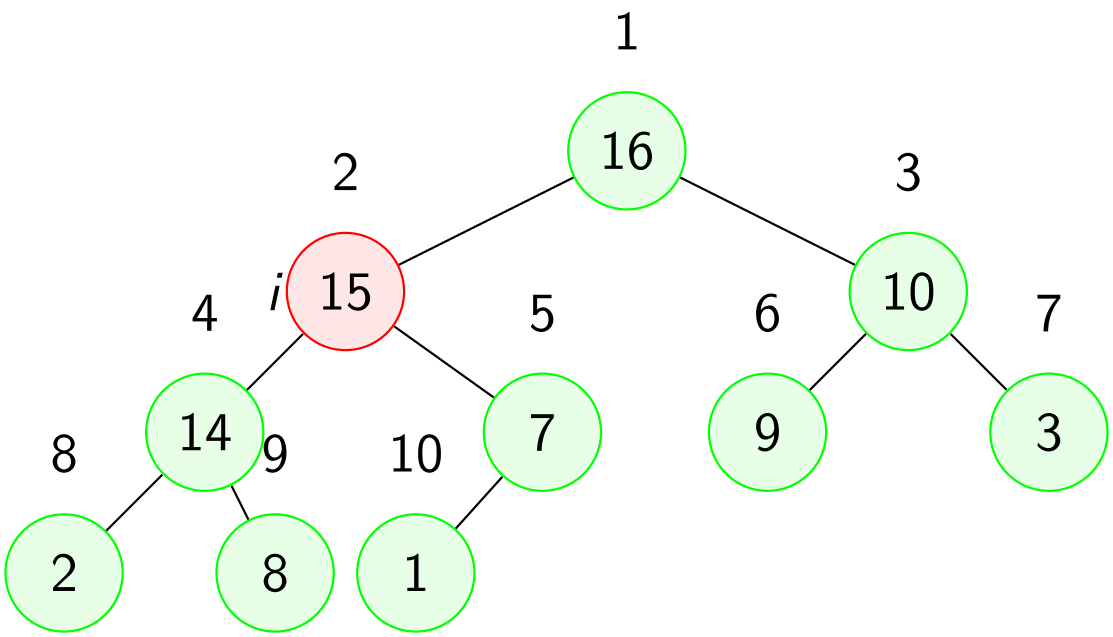
HEAP-INCREASE-KEY (A, i, 15)



HEAP-INCREASE-KEY (A, i, 15): exchange with the parent



HEAP-INCREASE-KEY (A, i, 15): exchange with the parent (contd.)



COMPLETE!!

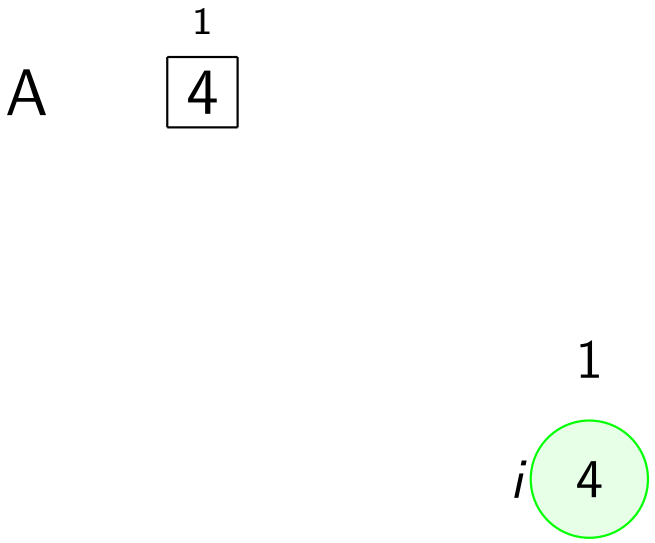
MAX-HEAP-INSERT

MAX-HEAP-INSERT(A, *key*)

1. A.*heap-size* $=$ A.*heap-size* $+ 1$
2. A[A.*heap-size*] $=$ -∞
3. HEAP-INCREASE-KEY (A, A.*heap-size*, *key*)

MAX-HEAP-INSERT has a running time of $\mathcal{O}(logn)$

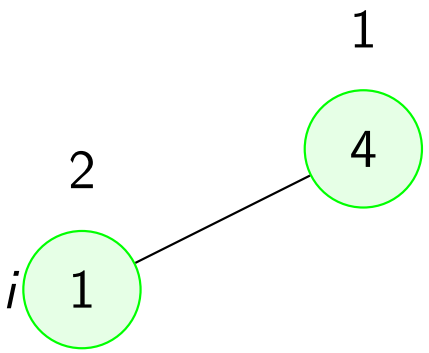
MAX-HEAP-INSERT(A, 4)



MAX-HEAP-INSERT(A, 1)

A

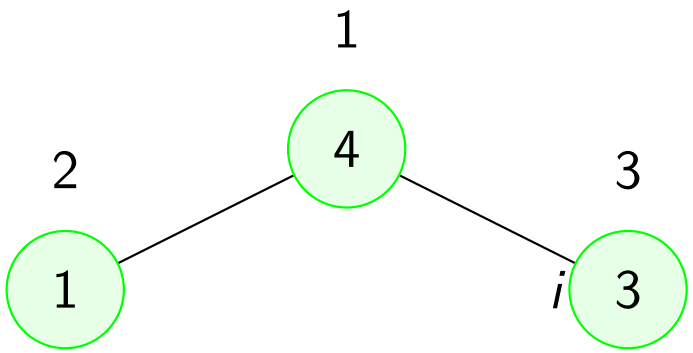
1	2
4	1



MAX-HEAP-INSERT(A, 3)

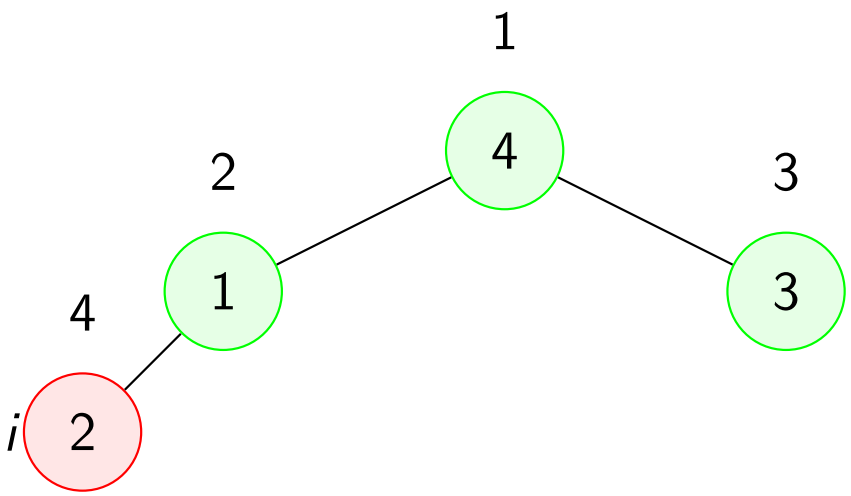
A

1	2	3
4	1	3



MAX-HEAP-INSERT(A, 2)

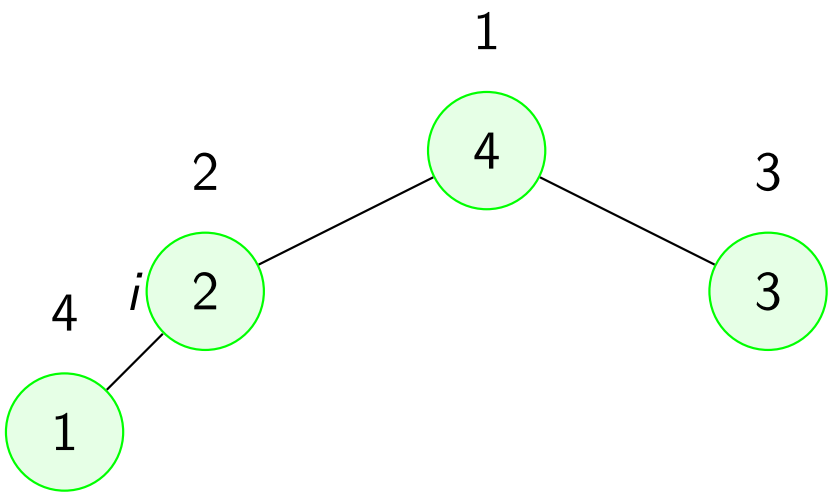
	1	2	3	4
A	4	1	3	2



MAX-HEAP-INSERT(A, 2): exchange A[i] with the parent

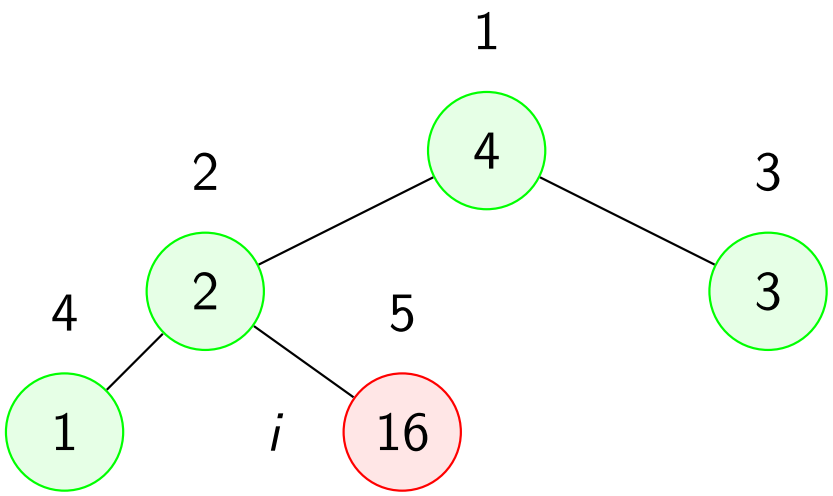
A

1	2	3	4
4	2	3	1



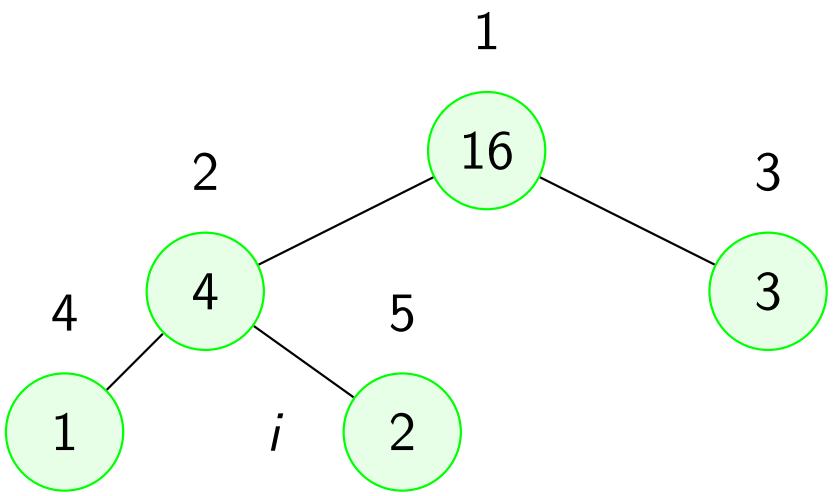
MAX-HEAP-INSERT(A, 16)

	1	2	3	4	5
A	4	2	3	1	16



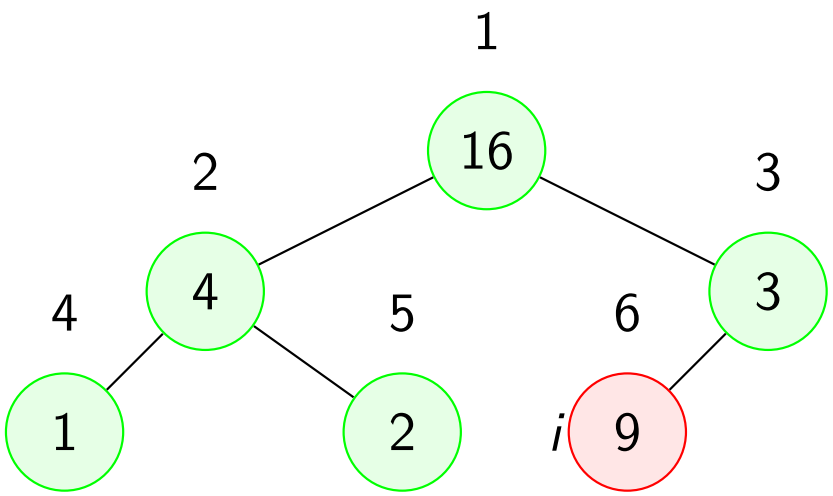
MAX-HEAP-INSERT(A, 16)

	1	2	3	4	5
A	16	4	3	1	2



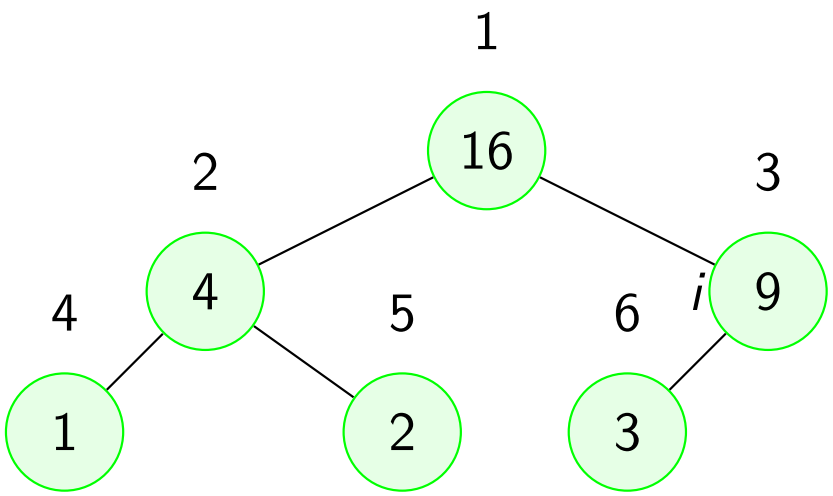
MAX-HEAP-INSERT(A, 9)

	1	2	3	4	5	6
A	16	4	3	1	2	9



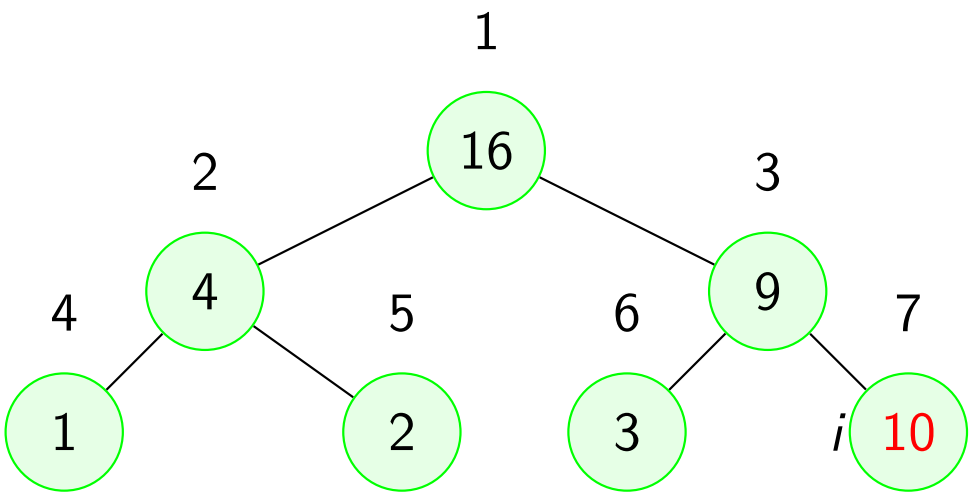
MAX-HEAP-INSERT(A, 9): exchange A[i] with parent

	1	2	3	4	5	6
A	16	4	9	1	2	3



MAX-HEAP-INSERT(A, 10)

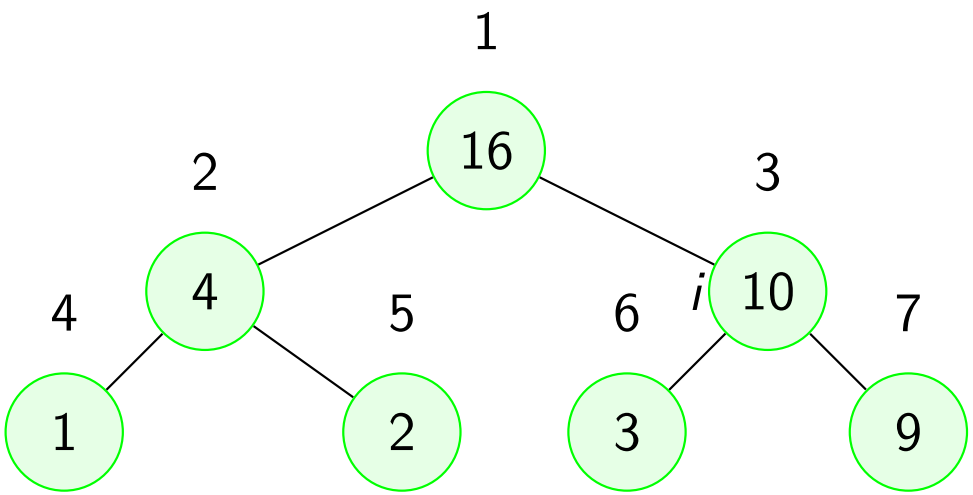
	1	2	3	4	5	6	7
A	16	4	9	1	2	3	10



MAX-HEAP-INSERT(A, 10): exchange A[i] with parent

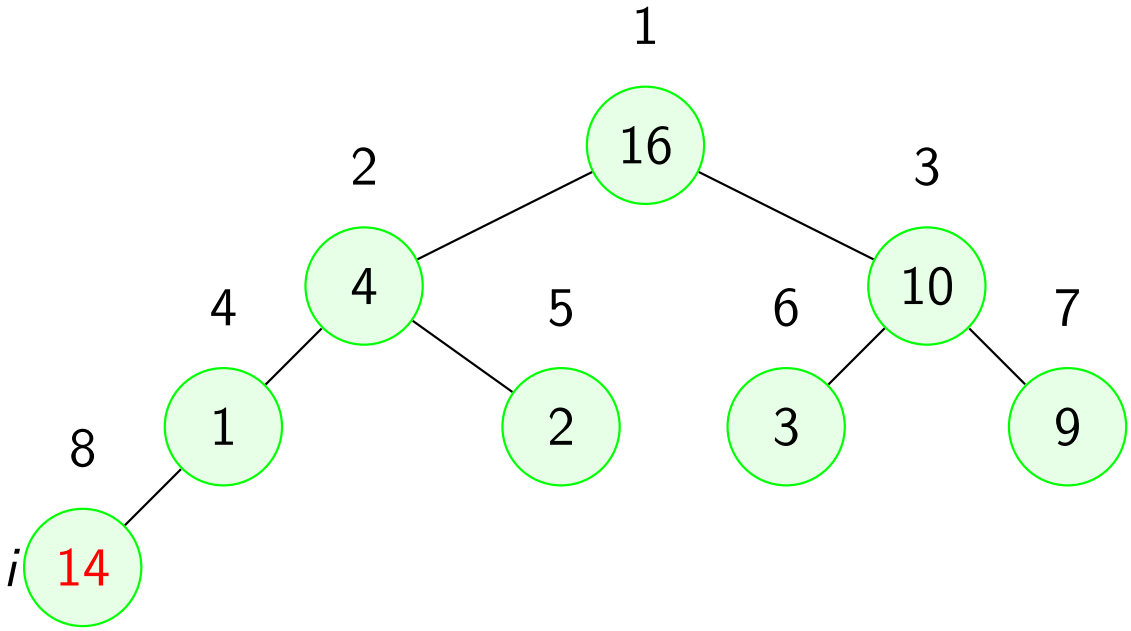
A

1	2	3	4	5	6	7
16	4	10	1	2	3	9



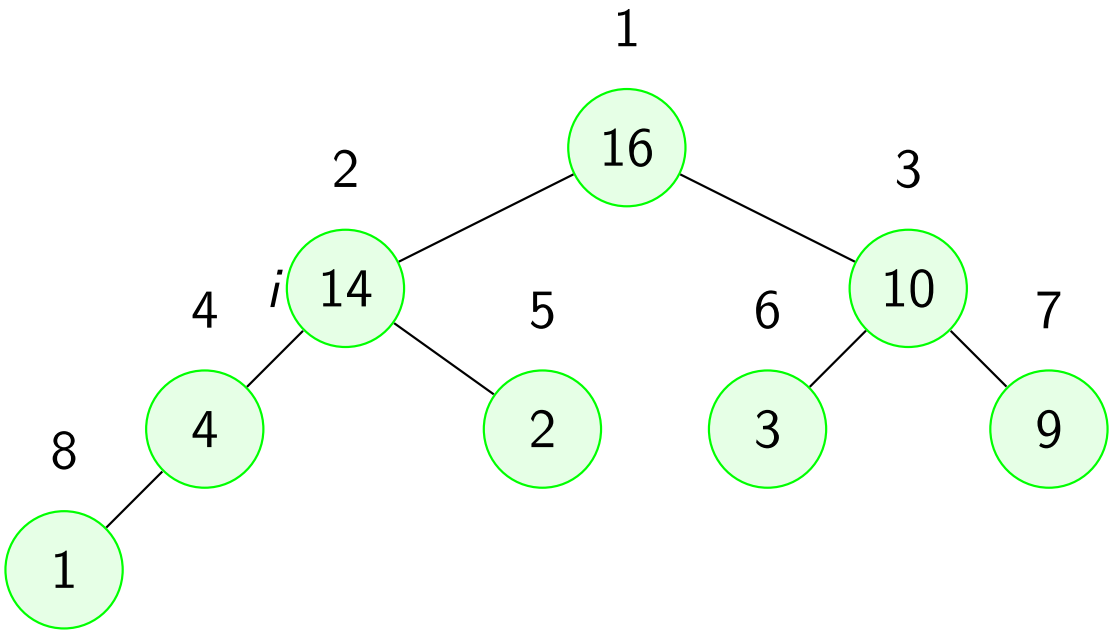
MAX-HEAP-INSERT(A, 14)

	1	2	3	4	5	6	7	8
A	16	4	10	1	2	3	9	14



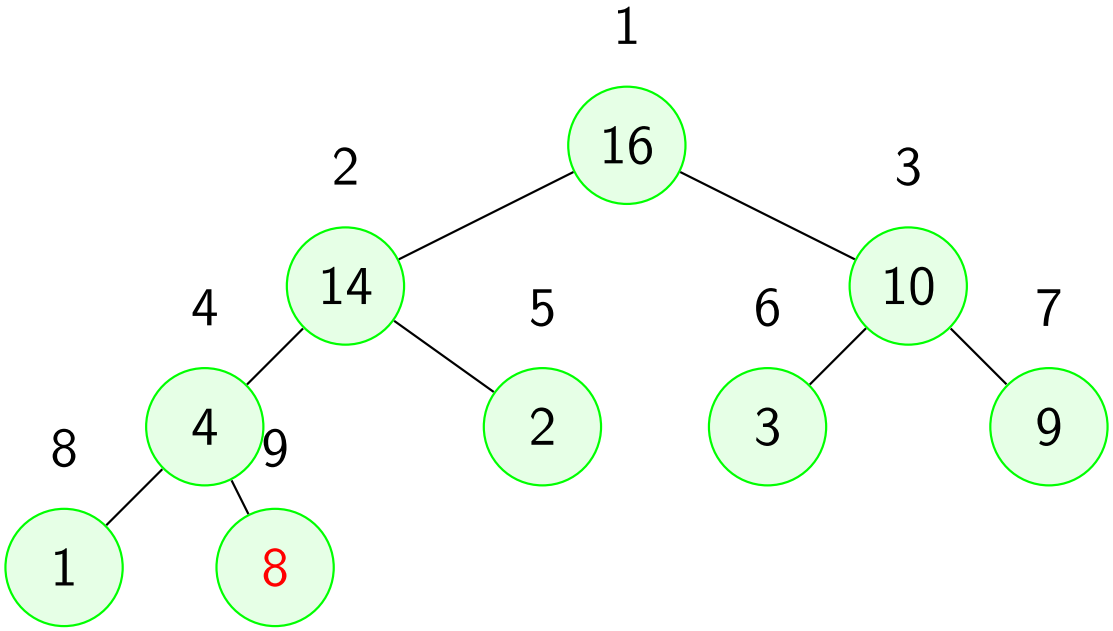
MAX-HEAP-INSERT(A, 14): exchange A[i] with parent (twice)

	1	2	3	4	5	6	7	8
A	16	14	10	4	2	3	9	1



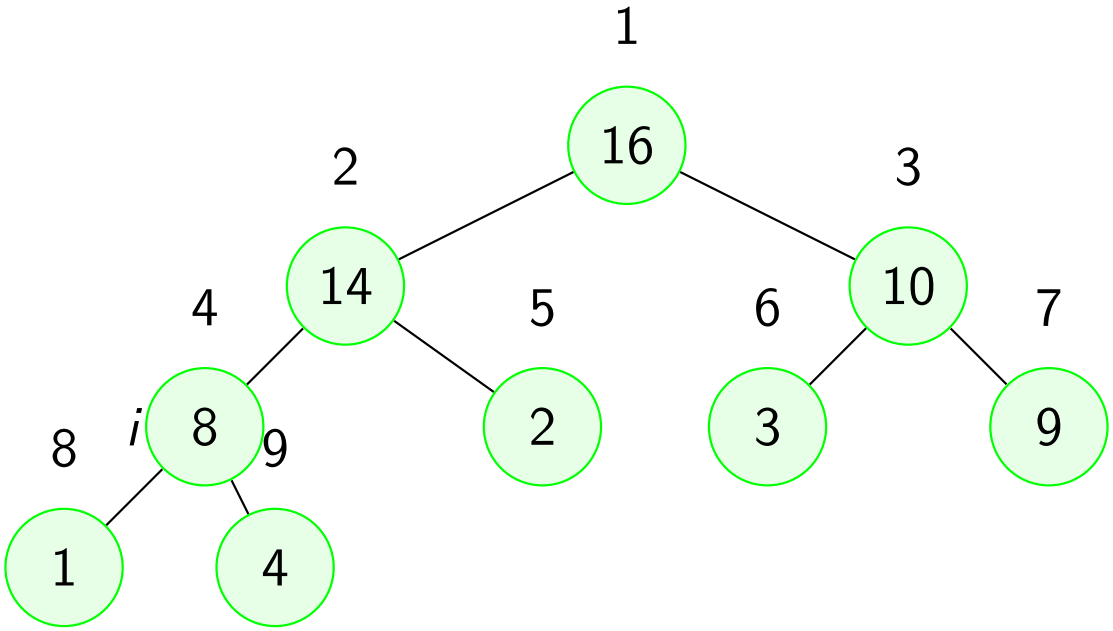
MAX-HEAP-INSERT(A, 8)

	1	2	3	4	5	6	7	8	9
A	16	14	10	4	2	3	9	1	8



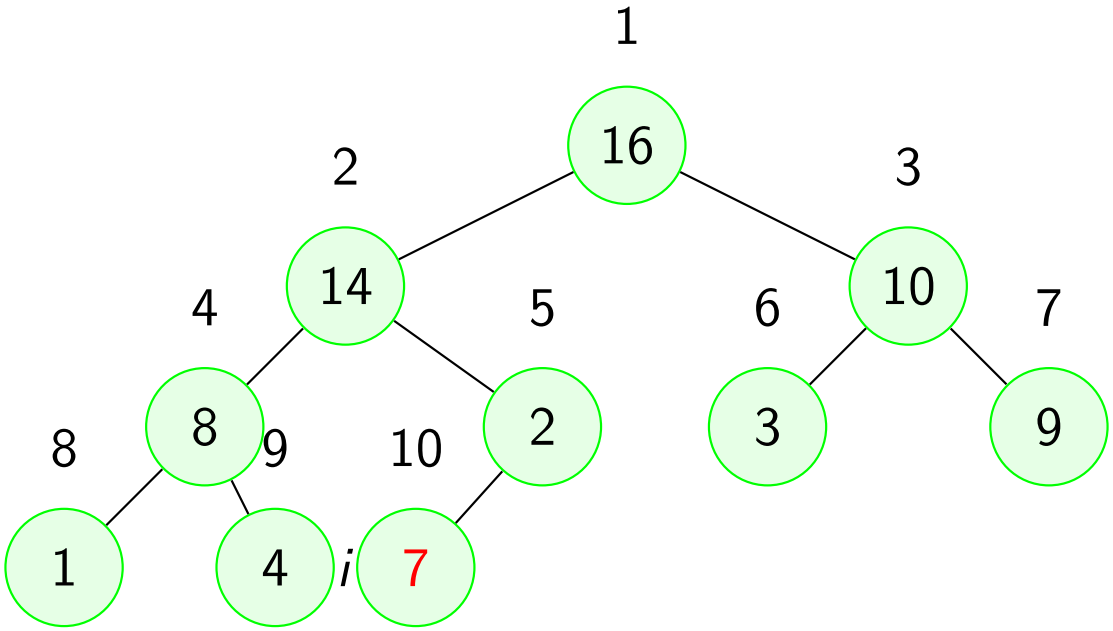
MAX-HEAP-INSERT(A, 8): exchange A[i] with parent

	1	2	3	4	5	6	7	8	9
A	16	14	10	8	2	3	9	1	4



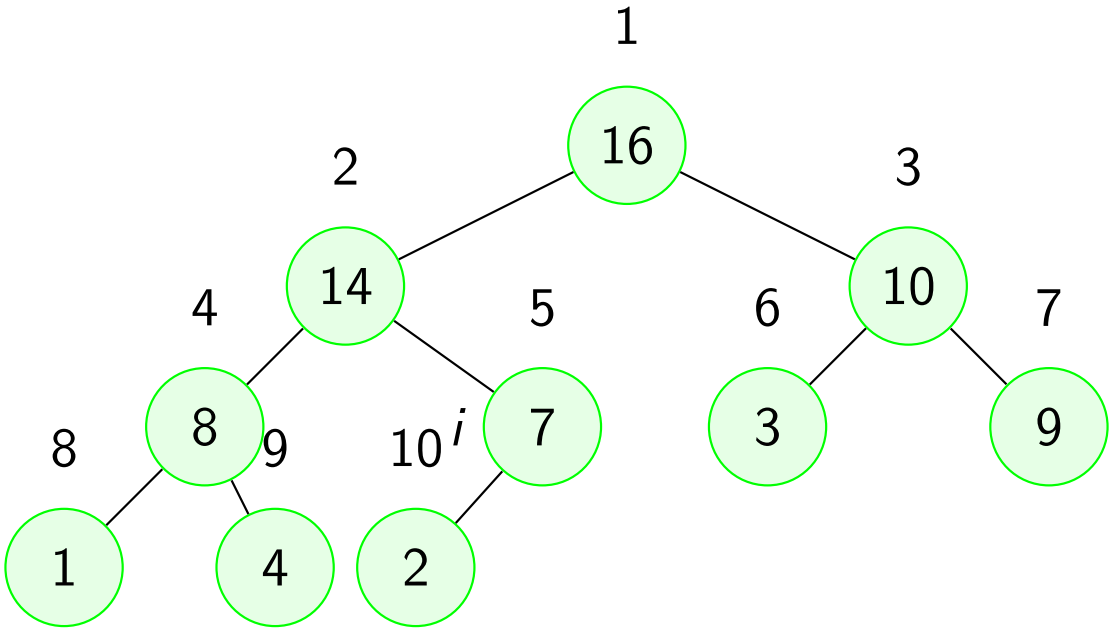
MAX-HEAP-INSERT(A, 7)

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	2	3	9	1	4	7



MAX-HEAP-INSERT(A, 7): exchange A[i] with parent

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	7	3	9	1	4	2



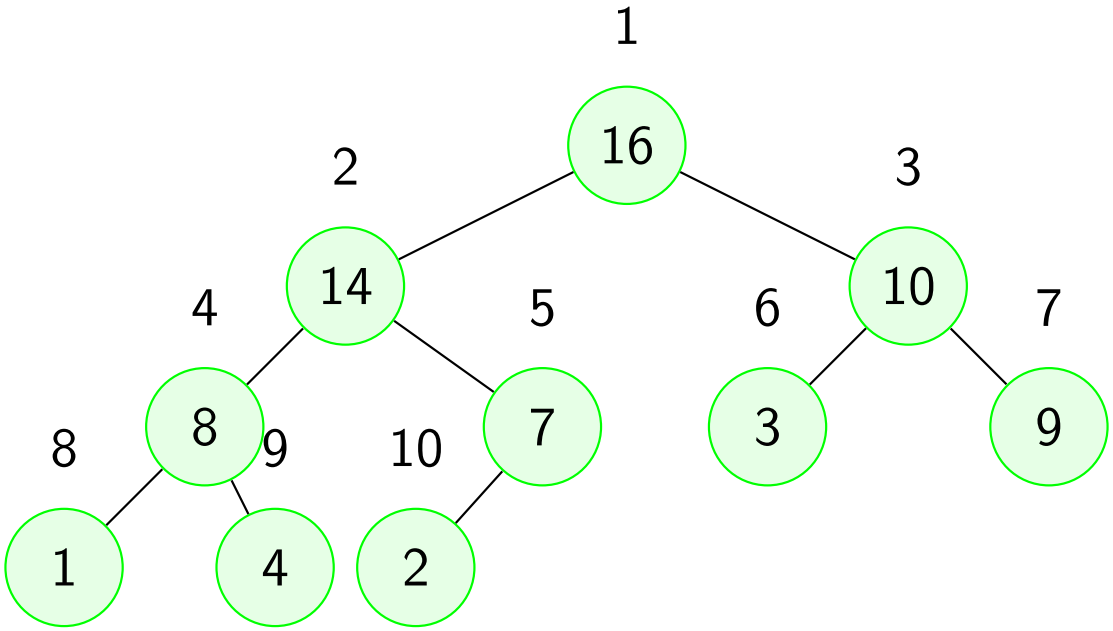
EXTRACT-MAX(S) (again)

HEAP-EXTRACT-MAX(S)

1. if A.*heap-size* < 1
2. error "heap overflow"
3. $max = A[1]$
4. $A[1] = A[\ A.heap\text{-}size]$
5. A.*heap-size* $=$ A.*heap-size* - 1
6. MAX-HEAPIFY(A,1)
7. return *max*

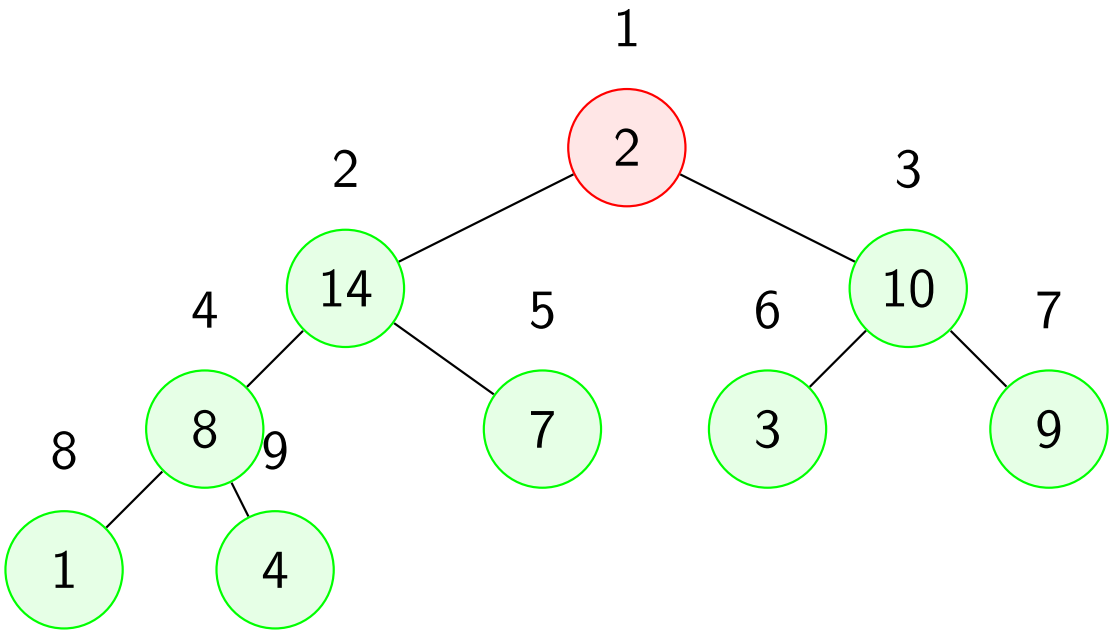
EXTRACT-MAX(A): maximum identified

	1	2	3	4	5	6	7	8	9	10
A	16	14	10	8	7	3	9	1	4	2



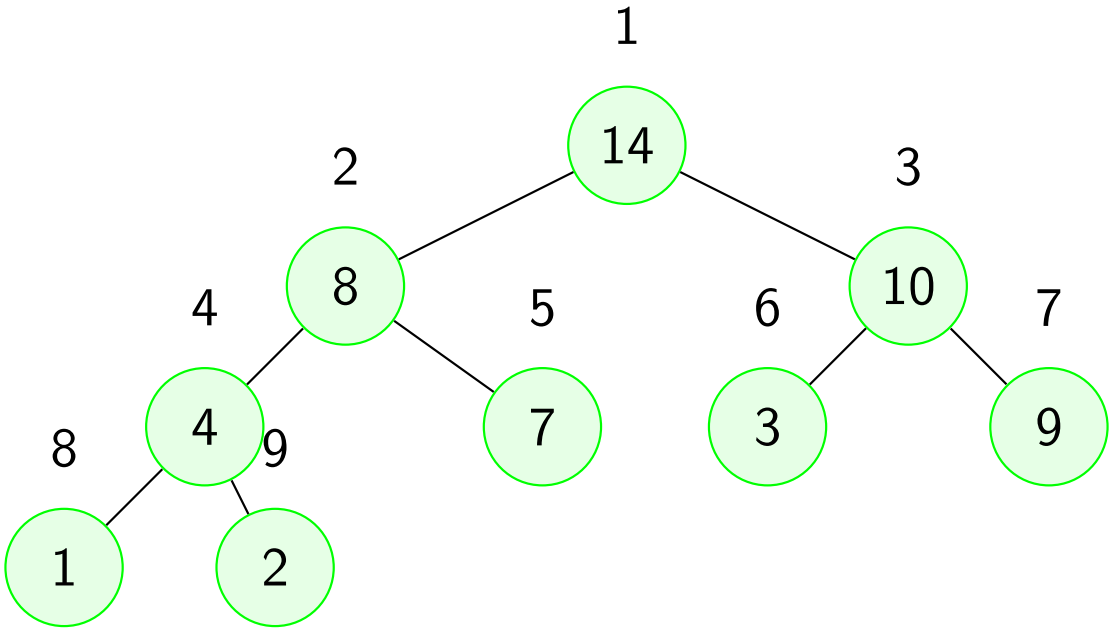
EXTRACT-MAX(A): A[1] replaced by the last element, heapsize reduced

	1	2	3	4	5	6	7	8	9
A	2	14	10	8	7	3	9	1	4



EXTRACT-MAX(A): MAX-HEAPIFY(A,1)

	1	2	3	4	5	6	7	8	9
A	14	8	10	4	7	3	9	1	2



Basic implementations

Right child

```
//Function to get right child of a node of a tree
int get_right_child(int A[], int index)
{
   if((((2*index) + 1) < tree_array_size) && (index >= 1)) May also use
(2*index) + 2 if array index starts from 0
      return (2*index) + 1;
   return -1;
}
```

Basic implementations (contd.)

Left child

```
//Function to get left child of a node of a tree
int get_left_child(int A[], int index)
{
   if(((2*index) < tree_array_size) && (index >= 1)) May also use (2*index) + 1
if array index starts from 0
      return 2*index;
   return -1;
}
```

Basic implementations (contd.)

Parent

```
//Function to get parent of a node of a tree
int get_parent(int A[], int index)
{
   if ((index > 1) && (index < tree_array_size))
   {
      return index/2;
   }
   return -1;
}
```

Basic implementations (contd.)

Insert into a heap

```
void insert(int A[], int key)
{
   heap_size++;
   A[heap_size] = -1*INF;
   increase_key(A, heap_size, key);
}
```

Basic implementations (contd.)

Increase key (max heap)

```
void increase_key(int A[], int index, int key)
{
  A[index] = key;
  while((index>1) && (A[get_parent(A, index)] < A[index])) Checking for the violation of (max) heap property
  {
    swap(&A[index], &A[get_parent(A, index)]);
    index = get_parent(A, index);
  }
}
```

Basic implementations (contd.)

Extract max

```
int extract_max(int A[])
{
   int maxm = A[1]; //A[0] if array index starts with 0
   A[1] = A[heap_size];
   heap_size– –;
   max_heapify(A, 1);
   return maxm;
}
```

THANK YOU !!!